reusable-extensible-python-functions-financial-data-analysis

November 1, 2026

0.1 Post Updates

Update 11/1/2026: Added `backtest_dca_strategy` and `calc_money_weighted_return`, the functions behind the dollar cost averaging post.

Update 10/31/2026: Added `add_rsi_ma_bb`, `analyze_trades`, `backtest_rsi_multi_asset_strategy`, `calculate_rsi`, `compute_daily_performance`, `create_signals`, `filter_top_percent`, `load_crypto_data`, `plot_iteration_results`, and `plot_multi_asset_equity_and_drawdown`, the functions behind the trend following and RSI crypto strategy posts.

0.2 Introduction

This post intends to provide the code for all of the python functions that I use in my analysis. The goal here is that when writing another post I will simply be able to link to the functions below as opposed to providing the function code in each post.

0.3 Python Imports

[1]:
```
# Standard Library
import os
import re
import sys
import warnings

from pathlib import Path

# Python Modules
from IPython.display import Markdown, display

# Suppress warnings
warnings.filterwarnings("ignore")
```

0.4 Add Directories To Path

[2]:
```
# Add the source subdirectory to the system path to allow import config from␣
↪settings.py
current_directory = Path(os.getcwd())
website_base_directory = current_directory.parent.parent.parent
```

```
src_directory = website_base_directory / "src"
sys.path.append(str(src_directory)) if str(src_directory) not in sys.path else␣
 ↪None

# Import settings.py
from settings import config

# Add configured directories from config to path
SOURCE_DIR = config("SOURCE_DIR")
sys.path.append(str(Path(SOURCE_DIR))) if str(Path(SOURCE_DIR)) not in sys.path␣
 ↪else None
```

0.5 Function Index

- add_rsi_ma_bb: Add RSI, moving averages, and Bollinger Bands for each crypto asset.
- analyze_trades: Summarize a set of trades: win rate, returns, PnL, and maximum drawdown.
- backtest_rsi_multi_asset_strategy: Backtest an RSI entry strategy across one or more assets, with limit or market entries, a trailing stop exit, and trading fees.
- backtest_target_weight_portfolio: Backtest a portfolio that periodically rebalances toward a set of target weights.
- bb_clean_data: Takes an Excel export from Bloomberg, removes the miscellaneous headings/rows, and returns a DataFrame.
- build_index: Reads the `index_temp.md` markdown file, inserts the markdown dependencies where indicated, and then saves the file as `index.md`.
- calc_fed_cycle_asset_performance: Calculates metrics for an asset based on a specified Fed tightening/easing cycle.
- calc_vix_trade_pnl: Calculates the profit/loss from VIX options trades.
- calculate_rsi: Calculate the Relative Strength Index for a price series using Wilder's smoothing.
- coinbase_fetch_available_products: Fetch available products from Coinbase Exchange API.
- coinbase_fetch_full_history: Fetch full historical data for a given product from Coinbase Exchange API.
- coinbase_fetch_historical_candles: Fetch historical candle data for a given product from Coinbase Exchange API.
- coinbase_pull_data: Update existing record or pull full historical data for a given product from Coinbase Exchange API.
- compute_daily_performance: Build the daily equity, cash, position, return, and drawdown series from a set of trades.
- create_ma_trend_signals: Generate multi-asset trend following target weights from a grid of moving averages.
- create_signals: Generate entry signals combining RSI, an optional moving average filter, and optional Bollinger Bands.
- df_info: A simple function to display the information about a DataFrame and the first five rows and last five rows.
- df_info_markdown: Similar to the `df_info` function above, except that it coverts the output to markdown.

- export_track_md_deps: Exports various text outputs to markdown files, which are included in the `index.md` file created when building the site with Hugo.
- filter_top_percent: Filter a DataFrame to the rows in the top X% for one or more columns.
- load_crypto_data: Load minute-level data for one or more crypto assets from Coinbase and merge it into a single DataFrame.
- load_data: Load data from a CSV, Excel, or Pickle file into a pandas DataFrame.
- pandas_set_decimal_places: Set the number of decimal places displayed for floating-point numbers in pandas.
- plot_bar_returns_ffr_change: Plot the bar chart of the cumulative or annualized returns for the asset class along with the change in the Fed Funds Rate.
- plot_histogram: Plot the histogram of a data set from a DataFrame.
- plot_iteration_results: Plot the distribution of backtest parameter sweep results, and the MAR ratio against each parameter and statistic.
- plot_multi_asset_equity_and_drawdown: Plot equity, cumulative return, drawdown, and each asset's price history with trades.
- plot_scatter: Plot the data from a DataFrame for a specified date range and columns.
- plot_scatter_regression_ffr_vs_returns: Plot a scatter plot of annualized returns vs annualized change in Fed Funds Rate, with a regression line and annotations for each policy cycle.
- plot_stats: Generate a scatter plot for the mean OHLC prices.
- plot_time_series: Plot the time series data from a DataFrame for a specified date range and columns.
- plot_vix_with_trades: Plot the VIX daily high and low prices, along with the VIX spikes, and trades.
- polygon_fetch_full_history: Fetch full historical data for a given product from Polygon API.
- polygon_fetch_ticker_details: Fetch detailed information for a given product from Polygon API.
- polygon_pull_data: Read existing data file, download price data from Polygon, and export data.
- run_regression: Run a regression using a variety of different methods and return the results.
- strategy_harry_brown_perm_port: Execute the strategy for the Harry Brown permanent portfolio.
- summary_stats: Generate summary statistics for a series of returns.
- yf_pull_data: Download daily price data from Yahoo Finance and export it.

0.6 Python Functions

0.6.1 add_rsi_ma_bb

```
import pandas as pd

from calculate_rsi import calculate_rsi


def add_rsi_ma_bb(
    tickers: list,
    data: pd.DataFrame,
    rsi_period: int,
```

```
    ma_days: list,
    bb_window: int,
    bb_num_std: float,
) -> pd.DataFrame:
    """
    Adds RSI, moving averages,and Bollinger bands for each crypto asset.

    Parameters
    ----------
    tickers : list
        List of crypto tickers, e.g., ["BTC-USD", "ETH-USD", "SOL-USD"].
    data : pd.DataFrame
        DataFrame containing merged price data for all tickers.
    rsi_period : int
        RSI lookback period.
    ma_days : list
        List of moving average durations in days.

    Returns
    -------
    pd.DataFrame
        DataFrame containing merged data for all tickers with RSI, moving averages,
        and bollinger bands.
    """

    # Copy the data dataframe
    df = data.copy()

    for ticker in tickers:
        # Shift close by 1 row
        df[f"{ticker}_close_prev"] = df[f"{ticker}_close"].shift(1)

        # ----- RSI -----

        # Calc RSI and shift by 1 row
        df[f"{ticker}_RSI"] = calculate_rsi(df[f"{ticker}_close"], period=rsi_period)
        df[f"{ticker}_RSI_prev"] = df[f"{ticker}_RSI"].shift(1)

        # ----- Moving Averages -----

        # Calc moving averages and shift by 1 row
        for day in ma_days:
            window = 1440 * day  # 1440 minutes in a day
            df[f"{ticker}_MA_{day}d"] = (
                df[f"{ticker}_close"].rolling(window=window, min_periods=1).mean()
            )
            df[f"{ticker}_MA_{day}d_prev"] = df[f"{ticker}_MA_{day}d"].shift(1)
```

```
        # ----- Bollinger Bands -----
        rolling = df[f"{ticker}_close_prev"].rolling(
            window=bb_window, min_periods=bb_window
        )
        df[f"{ticker}_BB_MID_prev"] = rolling.mean()
        df[f"{ticker}_BB_STD_prev"] = rolling.std()
        df[f"{ticker}_BB_UPPER_prev"] = df[f"{ticker}_BB_MID_prev"] + (
            bb_num_std * df[f"{ticker}_BB_STD_prev"]
        )
        df[f"{ticker}_BB_LOWER_prev"] = df[f"{ticker}_BB_MID_prev"] - (
            bb_num_std * df[f"{ticker}_BB_STD_prev"]
        )
        df[f"{ticker}_BB_Z_prev"] = (
            df[f"{ticker}_close_prev"] - df[f"{ticker}_BB_MID_prev"]
        ) / df[f"{ticker}_BB_STD_prev"]

    return df
```

0.6.2 analyze_trades

```
import pandas as pd


def analyze_trades(
    trades_df: pd.DataFrame,
    daily_perf_df: pd.DataFrame,
    print_summary: bool,
) -> tuple[int, float, float, float, float, float, float, float, float, float, float]:

    # Filter trades to only those that traded an actual position
    filtered_trades = trades_df[trades_df["quantity"] > 0.01]

    total_trades = len(filtered_trades)
    win_rate = len(filtered_trades[filtered_trades["return"] > 0]) / len(
        filtered_trades
    )
    total_return = daily_perf_df["Cum_Return"].iloc[-1]
    average_return_per_trade = filtered_trades["return"].mean()
    max_trade_gain_return = filtered_trades["return"].max()
    max_trade_loss_return = filtered_trades["return"].min()
    total_pnl = filtered_trades["pnl"].sum()
    average_pnl_per_trade = filtered_trades["pnl"].mean()
    max_trade_gain_pnl = filtered_trades["pnl"].max()
    max_trade_loss_pnl = filtered_trades["pnl"].min()
    max_drawdown = daily_perf_df["Drawdown"].min() * 100

    if print_summary:
        print("\nTrade Summary:")
```

```
        print(f"Total Trades: {total_trades}")
        print(f"Win Rate (%): {win_rate * 100:.2f}%")
        print(f"Total Return (%): {total_return * 100:.2f}%")
        print(f"Average Return Per Trade: {average_return_per_trade * 100:.2f}%")
        print(f"Max Trade Gain (%): {max_trade_gain_return * 100:.2f}%")
        print(f"Max Trade Loss (%): {max_trade_loss_return * 100:.2f}%")
        print(f"Total PnL ($): ${total_pnl:,.2f}")
        print(f"Average PnL Per Trade ($): ${average_pnl_per_trade:,.2f}")
        print(f"Max Trade Gain ($): ${max_trade_gain_pnl:,.2f}")
        print(f"Max Trade Loss ($): ${max_trade_loss_pnl:,.2f}")
        print(f"Max Drawdown (%): {max_drawdown:.2f}%")

    # Return all metrics as a tuple
    return (
        total_trades,
        win_rate,
        total_return,
        average_return_per_trade,
        max_trade_gain_return,
        max_trade_loss_return,
        total_pnl,
        average_pnl_per_trade,
        max_trade_gain_pnl,
        max_trade_loss_pnl,
        max_drawdown,
    )
```

0.6.3 backtest_dca_strategy

```
import numpy as np
import pandas as pd


def backtest_dca_strategy(
    fund: str,
    price_df: pd.DataFrame,
    price_column: str,
    contribution: float,
    drawdown_threshold: float | None,
    cash_yield: float | pd.Series,
    start_date: str,
    excel_export: bool,
    pickle_export: bool,
    output_confirmation: bool,
) -> pd.DataFrame:
    """
    Backtest dollar cost averaging into a single fund, optionally only buying
    while the fund is in a drawdown.
```

```
A fixed contribution arrives in cash on the last trading day of each month.
With no threshold, it is invested at that day's close. With a threshold, the
entire cash balance is invested at that day's close only if the fund's
drawdown is deeper than the threshold, and otherwise stays in cash earning
``cash_yield``. Shares are never sold.

The backtest starts on the first month end on or after ``start_date``, which
is the first day the portfolio has any value, and treats the final row of
``price_df`` as a month end. Trim a partial trailing month before calling, or
the final row will receive a contribution early.

Notes
-----
The buy decision uses the same close the trade executes at, the month end
convention also used by ``create_ma_trend_signals`` with ``signal_lag=0``.
There are no transaction costs.

Parameters
----------
fund : str
    Fund to invest in, e.g. ``"SPY"``.
price_df : pd.DataFrame
    Date-indexed DataFrame with a daily price column named
    ``f"{fund}_{price_column}"``. Include history before ``start_date``, since
    the drawdown is measured from the running peak of the full series.
price_column : str
    Suffix of the price column, e.g. ``"Adj_Close"``. With a total return
    series such as adjusted close, dividends are implicitly reinvested.
contribution : float
    Dollars added to cash on the last trading day of each month.
drawdown_threshold : float | None
    Drawdown required to buy, as a positive decimal, e.g. ``0.10`` buys only
    when the fund is more than 10% below its peak. ``None`` buys every month.
cash_yield : float | pd.Series
    Annual interest rate earned on cash, in decimal form. Pass a float for a
    flat rate or a date-indexed Series of annual rates. A Series is reindexed
    onto the backtest calendar and forward filled. Interest accrues on the
    balance carried in from the prior row, over the calendar days elapsed on
    a 365 day basis.
start_date : str
    Start date for the backtest in ``"YYYY-MM-DD"`` format.
excel_export : bool
    If True, export data to Excel format.
pickle_export : bool
    If True, export data to Pickle format.
output_confirmation : bool
    If True, print confirmation message.
```

```
    Returns
    -------
    pd.DataFrame
        Date-indexed DataFrame containing ``Price``, ``Drawdown``, ``Month_End``,
        ``Contribution``, ``Cash_Interest``, ``Buy``, ``Purchase``, ``Shares``,
        ``Invested``, ``Cash``, ``Total_Value``, ``Cash_%``,
        ``Cum_Contributions``, ``Cum_Cash_Interest``, ``Cum_Purchases``,
        ``Return`` and ``Cumulative_Return``.

        ``Return`` is net of that day's contribution,
        ``(Total_Value - Contribution) / prior Total_Value - 1``, so that a
        deposit does not count as a gain.

    Raises
    ------
    ValueError
        If ``contribution`` is not positive, ``drawdown_threshold`` is negative,
        the price column is missing, or no month end falls on or after
        ``start_date``.
    """

    if contribution <= 0:
        raise ValueError(f"  contribution must be > 0, got {contribution}.")

    if drawdown_threshold is not None and drawdown_threshold < 0:
        raise ValueError(
            f"  drawdown_threshold must be >= 0 or None, got {drawdown_threshold}."
        )

    column = f"{fund}_{price_column}"
    if column not in price_df.columns:
        raise ValueError(f"  price_df is missing required column: {column}")

    prices = price_df[column].dropna().sort_index()

    # Measure the drawdown before trimming, so that earlier peaks still count
    drawdown = prices / prices.cummax() - 1

    # Mark the last row of each month, treating the final row as a month end
    month = prices.index.to_period("M")
    month_end = pd.Series(
        np.append(month[:-1] != month[1:], True),
        index=prices.index,
    )

    # Start on the first month end, since nothing is held before it
    eligible = month_end[(month_end.index >= start_date) & month_end]
```

```
if eligible.empty:
    raise ValueError(
        f"  No month end on or after start_date {start_date}. "
        f"Price data ends {prices.index.max()}."
    )
first_date = eligible.index[0]

prices = prices[prices.index >= first_date]
drawdown = drawdown[drawdown.index >= first_date]
month_end = month_end[month_end.index >= first_date]

num_rows = prices.shape[0]
price_arr = prices.to_numpy(dtype=float)
drawdown_arr = drawdown.to_numpy(dtype=float)
month_end_arr = month_end.to_numpy(dtype=bool)

# Annual cash rate per row, either flat or tracking a supplied short rate
if isinstance(cash_yield, pd.Series):
    cash_rate = (
        cash_yield.reindex(prices.index).ffill().fillna(0).to_numpy(dtype=float)
    )
else:
    cash_rate = np.full(num_rows, float(cash_yield))

# The first row has no prior balance to accrue on, so its elapsed count is zero
days_elapsed = (
    prices.index.to_series().diff().dt.days.fillna(0).to_numpy(dtype=float)
)
interest_factor = cash_rate * days_elapsed / 365

# Output arrays
contributions = np.where(month_end_arr, float(contribution), 0.0)
cash_interest = np.zeros(num_rows)
buy = np.zeros(num_rows, dtype=bool)
purchase = np.zeros(num_rows)
shares = np.zeros(num_rows)
cash = np.zeros(num_rows)

prev_shares = 0.0
prev_cash = 0.0

for i in range(num_rows):
    interest = prev_cash * interest_factor[i]
    cash_in = prev_cash + interest + contributions[i]

    # Only month ends can trade, and the threshold must be strictly exceeded
    if month_end_arr[i] and (
        drawdown_threshold is None or drawdown_arr[i] < -drawdown_threshold
```

```
    ):
        buy[i] = True
        purchase[i] = cash_in

    shares[i] = prev_shares + purchase[i] / price_arr[i]
    cash[i] = cash_in - purchase[i]
    cash_interest[i] = interest

    prev_shares = shares[i]
    prev_cash = cash[i]

# Assemble the output frame in the documented column order
out = pd.DataFrame(index=prices.index)
out["Price"] = price_arr
out["Drawdown"] = drawdown_arr
out["Month_End"] = month_end_arr
out["Contribution"] = contributions
out["Cash_Interest"] = cash_interest
out["Buy"] = buy
out["Purchase"] = purchase
out["Shares"] = shares
out["Invested"] = shares * price_arr
out["Cash"] = cash
out["Total_Value"] = out["Invested"] + out["Cash"]
out["Cash_%"] = out["Cash"] / out["Total_Value"]
out["Cum_Contributions"] = out["Contribution"].cumsum()
out["Cum_Cash_Interest"] = out["Cash_Interest"].cumsum()
out["Cum_Purchases"] = out["Purchase"].cumsum()

# Performance, net of each day's contribution
out["Return"] = (out["Total_Value"] - out["Contribution"]) / out[
    "Total_Value"
].shift(1) - 1
out["Cumulative_Return"] = (1 + out["Return"]).cumprod()

threshold_label = (
    "Every_Month"
    if drawdown_threshold is None
    else f"{drawdown_threshold:.0%}_Drawdown"
)
plan_name = f"{fund}_DCA_{threshold_label}"

# Export to excel
if excel_export == True:
    out.to_excel(f"{plan_name}_Strategy.xlsx", sheet_name="data")
else:
    pass
```

```
    # Export to pickle
    if pickle_export == True:
        out.to_pickle(f"{plan_name}_Strategy.pkl")
    else:
        pass

    # Output confirmation
    if output_confirmation == True:
        print(
            f"Strategy complete for {plan_name}, "
            f"buying in {buy.sum()} of {month_end_arr.sum()} months"
        )
    else:
        pass

    return out
```

0.6.4 backtest_rsi_multi_asset_strategy

```
import numpy as np
import pandas as pd


def backtest_rsi_multi_asset_strategy(
    tickers: list,
    prices: pd.DataFrame,
    signals: pd.DataFrame,
    initial_capital: float,
    rsi_threshold: float,  # kept for signature compatibility (unused here)
    trailing_stop_pct: float,
    ma_days: list,  # kept for signature compatibility (unused here)
    order_entry: str,
    trading_fees: bool,
    trade_taker_fee: float,  # market order fee
    trade_maker_fee: float,  # limit order fee
) -> pd.DataFrame:
    """
    Optimized backtest with legacy 'no-exit leaves cash stuck' behavior:
    - If no exit is found from entry to the end of data, cash is NOT refunded and the
      trade remains open (matching your original implementation).
    - Avoids copying prices_df inside the loop; uses NumPy views for speed.
    """

    # Stable sort; shallow (meta-only) copy to avoid duplicating data
    prices_df = prices.copy(deep=False).sort_values(by="Date", kind="mergesort")
    date_idx = prices_df["Date"].to_numpy()

    # Create dictionary of prices as numpy arrays
```

```
prices_dict = {}
for t in tickers:
    prices_dict[t] = (
        prices_df[f"{t}_open"].to_numpy(),
        prices_df[f"{t}_high"].to_numpy(),
        prices_df[f"{t}_low"].to_numpy(),
    )

# Chronological signals; shallow copy to keep memory low
signals_sorted_df = signals.copy(deep=False).sort_values(
    by="Date", kind="mergesort"
)
signals_grouped_df = signals_sorted_df.groupby("Date", sort=True)

# Initialize tracking variables
cash = initial_capital

# Initialize trading fee
trade_entry_fee_dec = 0.0
trade_exit_fee_dec = 0.0

# Create list for trades
trades = []

# Start with the first timestamp
next_timestamp = prices_df["Date"].iloc[0]

# Iterate through signals DataFrame
for timestamp, signal_rows in signals_grouped_df:

    # Check if signal was during a previous trade
    if timestamp <= next_timestamp:
        continue

    # Iterate over each asset
    for signal in signal_rows.itertuples(index=False):

        # -------- ENTRY --------

        # Get the ticker
        ticker = getattr(signal, "asset")

        # Set the entry timestamp
        entry_timestamp = timestamp

        # Get the allocation
        allocation_pct = getattr(signal, "allocation_pct")
```

```
# Calc capital based on number of assets and allocation_pct
capital_to_use = cash / len(tickers) * allocation_pct

# # Shared capital, scaled by allocation
# capital_to_use = (cash / max(1, len(tickers))) * allocation_pct
# if capital_to_use <= 0:
#     continue

# Get entry price and trade entry fee percentage
# Entry on open for a market order
if order_entry == "market":
    entry_price = getattr(signal, "open")
    if trading_fees:
        trade_entry_fee_dec = trade_taker_fee
# Entry on a limit order set at previous candle close
elif order_entry == "limit":
    close_prev = getattr(signal, "close_prev")
    high = getattr(signal, "high")
    low = getattr(signal, "low")

    if close_prev >= low and close_prev <= high:
        entry_price = getattr(signal, "close_prev")
        if trading_fees:
            trade_entry_fee_dec = trade_maker_fee
    else:
        continue

# Calc entry value and entry fee
entry_value = capital_to_use / (1.0 + trade_entry_fee_dec)
entry_fee = capital_to_use - entry_value

# Calc quantity to buy
quantity = entry_value / entry_price

# Update cash position based on entry
cash -= entry_value
cash -= entry_fee

# -------- EXIT --------

# Extract numpy arrays
open_nparray, high_nparray, low_nparray = prices_dict[ticker]

# Find signal timestamp in prices date index, start is index value
start = np.searchsorted(
    date_idx, np.datetime64(entry_timestamp), side="left"
)
if start >= len(date_idx):
```

```
        # No bars remain; legacy behavior: keep cash debited, position stays open
        continue

    # Slices from the price numpy arrays
    o_sub = open_nparray[start:]
    h_sub = high_nparray[start:]
    l_sub = low_nparray[start:]
    d_sub = date_idx[start:]

    # Running peak (never below entry price)
    peak_price = np.maximum.accumulate(np.maximum(h_sub, entry_price))
    stop_price = peak_price * (1.0 - trailing_stop_pct)

    # Gap-breach first: open <= stop and low <= stop -> exit at open
    gap_idx = np.flatnonzero((l_sub <= stop_price) & (o_sub <= stop_price))

    # Regular breach: low <= stop -> exit at stop price
    br_idx = np.flatnonzero(l_sub <= stop_price)

    i_gap = int(gap_idx[0]) if gap_idx.size else np.inf
    i_br = int(br_idx[0]) if br_idx.size else np.inf

    if np.isinf(min(i_gap, i_br)):
        # ---- LEGACY QUIRK: no exit to end of data -> keep cash debited, stay open
        continue

    if i_gap <= i_br:
        # Gap-breach is earlier (or same bar) -> exit at open
        i = i_gap
        exit_timestamp = pd.Timestamp(d_sub[i].astype("datetime64[ns]"))
        exit_price = float(o_sub[i])
        order_exit = "exit at open (gap)"
    else:
        # Regular breach is earlier -> exit at stop level of that bar
        i = i_br
        exit_timestamp = pd.Timestamp(d_sub[i].astype("datetime64[ns]"))
        exit_price = float(stop_price[i])
        order_exit = "trailing stop"

    # Get trade exit fee percentage
    if trading_fees:
        trade_exit_fee_dec = trade_taker_fee

    # Calc exit value and exit fee
    exit_value = quantity * exit_price
    exit_fee = exit_value * trade_exit_fee_dec

    # Update cash position based on exit
```

```
            cash += exit_value
            cash -= exit_fee

            # Calc pnl, return
            pnl = (exit_value - exit_fee) - (entry_value + entry_fee)
            return_dec = pnl / capital_to_use

            # Add to trades list
            trades.append(
                {
                    "asset": ticker,
                    "entry_time": entry_timestamp,
                    "entry_type": order_entry,
                    "entry_price": entry_price,
                    "exit_time": exit_timestamp,
                    "exit_type": order_exit,
                    "exit_price": exit_price,
                    "quantity": quantity,
                    "allocation_pct": allocation_pct,
                    "pnl": pnl,
                    "return": return_dec,
                    "cash": cash,
                    "entry_fee": entry_fee,
                    "exit_fee": exit_fee,
                }
            )

            # Update next_timestamp
            next_timestamp = exit_timestamp

    # Create new dataframe for trades
    trades_df = pd.DataFrame(trades)

    # If there are entries, calc cumulative pnl, equity, cumulative return
    if not trades_df.empty:
        trades_df["cumulative_pnl"] = trades_df["pnl"].cumsum()
        trades_df["equity"] = trades_df["cumulative_pnl"] + initial_capital
        trades_df["cumulative_return"] = trades_df["equity"] / initial_capital - 1

    return trades_df
```

0.6.5 backtest_target_weight_portfolio

```
import numpy as np
import pandas as pd


def backtest_target_weight_portfolio(
```

```
    fund_list: list[str],
    price_df: pd.DataFrame,
    weights_df: pd.DataFrame,
    price_column: str,
    starting_cash: float,
    cash_contrib: float,
    cash_yield: float | pd.Series,
    borrow_spread: float,
    leverage: float,
    rebal_interval: int | str,
    trans_cost_pct: float,
    start_date: str,
    excel_export: bool,
    pickle_export: bool,
    output_confirmation: bool,
) -> pd.DataFrame:
    """
    Backtest a portfolio that periodically rebalances toward target weights.

    The portfolio holds each fund at its target weight, with any unallocated
    remainder sitting in cash. Positions drift with prices between rebalances and
    are only traded back to target on the rows ``rebal_interval`` selects, either
    every N rows, counted from the first row on or after ``start_date`` so the
    rebalance calendar does not depend on how much price history precedes the
    backtest, or on the last trading day of each month.

    Supplied weights are scaled by ``leverage`` before being applied. A leverage
    above 1 does not force borrowing: it raises the ceiling on gross exposure, and
    whether the portfolio actually borrows depends on how much of that ceiling the
    weights use. A strategy whose weights sum to 0.6 levered at 1.25 targets 0.75
    of the portfolio and still holds a quarter of it in cash. The same strategy at
    a moment when its weights sum to 1.0 targets 1.25 and borrows a quarter of
    equity. Cash simply goes negative when it happens, and negative balances
    accrue at ``cash_yield`` plus ``borrow_spread`` rather than earning interest.

    Each row is recorded twice: before action (BA) values reflect the positions
    carried in from the prior row marked at today's prices, and after action (AA)
    values reflect the positions held once any rebalance trade has taken place. On
    a non-rebalance row the two are identical and no transaction costs accrue.

    The engine is agnostic about where the weights come from, so any strategy that
    can express itself as a per-fund target weight can be run through it.

    Notes
    -----
    Weights are consumed as supplied apart from the ``leverage`` scaling. Lagging
    them so that the backtest does not act on same-day information is the caller's
    responsibility; ``create_ma_trend_signals`` does this via its ``signal_lag``
```

```
parameter.

Margin is modeled only as a financing cost. There is no maintenance
requirement, no margin call, and no forced liquidation, so a leverage high
enough to be called in practice will run to the end of the sample here
regardless. Treat results above roughly 2x as indicative rather than
achievable.

Parameters
----------
fund_list : list[str]
    List of funds in the portfolio, e.g. ``["IVV", "EFA", "EEM"]``.
price_df : pd.DataFrame
    Date-indexed DataFrame containing a price column per fund, named
    ``f"{fund}_{price_column}"``. When this is a total return series such as
    adjusted close, dividends are implicitly reinvested.
weights_df : pd.DataFrame
    Date-indexed DataFrame containing a target weight per fund, named
    ``f"{fund}_Target_Weight"``, on the same index as ``price_df``. Weights
    are expressed as a fraction of total portfolio value.
price_column : str
    Suffix of the price column to mark positions at, e.g. ``"Adj_Close"``.
starting_cash : float
    Starting portfolio balance, held as cash before the first rebalance.
cash_contrib : float
    Cash contribution added on every row. Contributions accumulate in cash
    and are deployed at the next rebalance.
cash_yield : float | pd.Series
    Annual interest rate earned on the uninvested cash balance, in decimal
    form. Pass a float for a flat rate across the whole backtest, or a
    date-indexed Series of annual rates to track an actual short rate such as
    the 3 month Treasury bill. A Series is reindexed onto the backtest
    calendar and forward filled. Pass ``0.0`` for no interest. Interest
    accrues on the balance carried in from the prior row, over the actual
    calendar days elapsed on a 365 day basis, and is credited before the
    rebalance decision so it is available to invest.
borrow_spread : float
    Annual markup over ``cash_yield`` charged on a negative cash balance, in
    decimal form. Broker margin is typically priced as a benchmark rate plus a
    spread, roughly 100 to 150 basis points for a retail account and far less
    institutionally. Pass ``0.0`` to borrow at the cash rate. Ignored while
    the cash balance stays positive.
leverage : float
    Multiplier applied to every supplied target weight. ``1.0`` leaves the
    weights untouched and the portfolio can never borrow. Above 1.0 the
    portfolio borrows only on rows where the scaled weights exceed 1.0; below
    1.0 it deliberately underinvests.
rebal_interval : int | str
```

```
    Rebalance calendar. An integer rebalances every N rows, counted from the
    first row of the backtest. ``"ME"`` instead rebalances on the last row of
    each month, which is the last trading day present in the data rather than
    the calendar month end, since that need not be a trading day. The first
    row of the backtest is always a rebalance under either setting, which is
    what deploys the starting cash. Under ``"ME"`` the final row of the
    backtest is never a rebalance: no row follows it for the trade to
    benefit, so it would pay transaction costs for nothing.
trans_cost_pct : float
    Transaction cost as a fraction of traded notional, e.g. ``0.001`` for 10
    basis points. Applied to the absolute value of each fund's trade.
start_date : str
    Start date for the backtest in ``"YYYY-MM-DD"`` format, or ``"All"`` to
    run from the first row of ``price_df``.
excel_export : bool
    If True, export data to Excel format.
pickle_export : bool
    If True, export data to Pickle format.
output_confirmation : bool
    If True, print confirmation message.

Returns
-------
pd.DataFrame
    Date-indexed DataFrame containing, per fund, the applied target weight
    (post-leverage, not as supplied), before action shares and dollars
    invested, after action shares, dollars invested, realized weight and
    traded notional, alongside the portfolio level ``Total_BA_$_Invested``,
    ``Cash_Interest``, ``Cash_BA``, ``Contribution``, ``Rebalance``,
    ``Total_AA_$_Invested``, ``Total_AA_%_Invested``,
    ``Total_Transaction_Costs``, ``Cash_AA``, ``Cash_%``, ``Borrowing``,
    ``Total_Value``, ``Return``, ``Cumulative_Return``,
    ``Cum_Total_Transaction_Costs`` and ``Cum_Cash_Interest``.

    ``Cash_Interest`` is signed: positive where the balance earned interest,
    negative where it was charged. ``Borrowing`` marks the rows where the
    portfolio was genuinely on margin.

Raises
------
ValueError
    If ``rebal_interval`` is neither a positive integer nor ``"ME"``,
    ``leverage`` is not positive, ``borrow_spread`` is negative, a required
    column is missing, or ``start_date`` falls beyond the end of the price
    data.
"""

if isinstance(rebal_interval, str):
```

```
    if rebal_interval != "ME":
        raise ValueError(
            f'  rebal_interval must be a positive int or "ME", '
            f"got {rebal_interval!r}."
        )
elif rebal_interval < 1:
    raise ValueError(f"  rebal_interval must be >= 1, got {rebal_interval}.")

if leverage <= 0:
    raise ValueError(f"  leverage must be > 0, got {leverage}.")

if borrow_spread < 0:
    raise ValueError(f"  borrow_spread must be >= 0, got {borrow_spread}.")

missing_prices = [
    f"{fund}_{price_column}"
    for fund in fund_list
    if f"{fund}_{price_column}" not in price_df.columns
]
if missing_prices:
    raise ValueError(f"  price_df is missing required columns: {missing_prices}")

missing_weights = [
    f"{fund}_Target_Weight"
    for fund in fund_list
    if f"{fund}_Target_Weight" not in weights_df.columns
]
if missing_weights:
    raise ValueError(
        f"  weights_df is missing required columns: {missing_weights}"
    )

# Align the two frames and trim to the backtest window
prices = price_df[[f"{fund}_{price_column}" for fund in fund_list]]
weights = weights_df[[f"{fund}_Target_Weight" for fund in fund_list]]
df = prices.join(weights, how="inner")

if start_date != "All":
    df = df[df.index >= start_date]

if df.empty:
    raise ValueError(
        f"  No rows remain for start_date {start_date}. "
        f"Price data ends {price_df.index.max()}."
    )

num_rows = df.shape[0]
num_funds = len(fund_list)
```

```
# Work in NumPy so the sequential path below stays fast
price_arr = df[[f"{fund}_{price_column}" for fund in fund_list]].to_numpy(
    dtype=float
)
# Scale the supplied weights. Gross exposure now runs up to leverage rather
# than up to 1, and cash absorbs the difference in either direction.
weight_arr = (
    df[[f"{fund}_Target_Weight" for fund in fund_list]].to_numpy(dtype=float)
    * leverage
)

# A fund only trades on rows where it has a price
tradable = np.isfinite(price_arr)
safe_price = np.where(tradable, price_arr, np.nan)

# Rebalance calendar. An integer interval is counted from the first row of
# the backtest; "ME" instead marks every row that its successor does not
# share a month with, the last trading day of the month as the data has it.
# That leaves the final row unflagged, which is deliberate: no row follows it
# for the trade to benefit, so it would pay transaction costs for nothing.
# Either way the first row rebalances, deploying the starting cash.
if isinstance(rebal_interval, str):
    month = df.index.to_period("M")
    rebal_flags = np.append(month[:-1] != month[1:], False)
    rebal_flags[0] = True
else:
    rebal_flags = np.arange(num_rows) % rebal_interval == 0

# Annual cash rate per row, either flat or tracking a supplied short rate
if isinstance(cash_yield, pd.Series):
    cash_rate = cash_yield.reindex(df.index).ffill().fillna(0).to_numpy(dtype=float)
else:
    cash_rate = np.full(num_rows, float(cash_yield))

# Interest accrues over the actual calendar days between rows. The first row
# has no prior balance to accrue on, so its elapsed count is zero. A positive
# balance earns the cash rate; a negative one is charged the cash rate plus
# the borrow spread, so which factor applies is decided inside the loop.
days_elapsed = df.index.to_series().diff().dt.days.fillna(0).to_numpy(dtype=float)
lend_factor = cash_rate * days_elapsed / 365
borrow_factor = (cash_rate + borrow_spread) * days_elapsed / 365

# Output arrays
ba_shares = np.zeros((num_rows, num_funds))
ba_invested = np.zeros((num_rows, num_funds))
aa_shares = np.zeros((num_rows, num_funds))
aa_invested = np.zeros((num_rows, num_funds))
```

```
notional = np.zeros((num_rows, num_funds))
total_ba_invested = np.zeros(num_rows)
cash_interest = np.zeros(num_rows)
cash_ba = np.zeros(num_rows)
cash_aa = np.zeros(num_rows)
total_aa_invested = np.zeros(num_rows)
total_costs = np.zeros(num_rows)
total_value = np.zeros(num_rows)

# Positions and cash carried into the current row
prev_shares = np.zeros(num_funds)
prev_cash = starting_cash

for i in range(num_rows):
    # ----- Before action -----

    # Positions carried in from the prior row, marked at today's prices
    shares_in = prev_shares
    invested_in = np.where(tradable[i], shares_in * safe_price[i], 0.0)
    ba_shares[i] = shares_in
    ba_invested[i] = invested_in
    total_ba_invested[i] = invested_in.sum()

    # Interest on the balance carried in, then this row's contribution.
    # Negative when the portfolio is borrowing rather than lending.
    if prev_cash >= 0:
        interest = prev_cash * lend_factor[i]
    else:
        interest = prev_cash * borrow_factor[i]
    cash_interest[i] = interest
    cash_in = prev_cash + interest + cash_contrib
    cash_ba[i] = cash_in

    # ----- After action -----

    portfolio_value = total_ba_invested[i] + cash_in

    if rebal_flags[i]:
        # Trade back to target weights, holding untradable funds flat.
        # Costs are paid out of cash, so the amount available to deploy is
        # the portfolio net of them. Since the cost depends on the size of
        # the trade and the trade depends on the amount deployed, price the
        # trade once at gross value, then redeploy against value net of that
        # first estimate. The residual is on the order of the cost squared.
        gross_dollars = np.nan_to_num(
            np.where(tradable[i], weight_arr[i] * portfolio_value, invested_in),
            nan=0.0,
        )
```

```
            cost_estimate = trans_cost_pct * np.abs(gross_dollars - invested_in).sum()

            target_dollars = np.nan_to_num(
                np.where(
                    tradable[i],
                    weight_arr[i] * (portfolio_value - cost_estimate),
                    invested_in,
                ),
                nan=0.0,
            )
            traded = target_dollars - invested_in
            costs = trans_cost_pct * np.abs(traded).sum()

            # Divide only where a price exists, holding other funds' shares flat
            divisor = np.where(tradable[i], safe_price[i], 1.0)
            aa_invested[i] = target_dollars
            aa_shares[i] = np.where(tradable[i], target_dollars / divisor, shares_in)
            notional[i] = traded
        else:
            # No trade, so after action simply mirrors before action
            aa_invested[i] = invested_in
            aa_shares[i] = shares_in
            notional[i] = 0.0
            costs = 0.0

        total_aa_invested[i] = aa_invested[i].sum()
        total_costs[i] = costs

        # Whatever is not invested, less costs, remains in cash
        cash_aa[i] = portfolio_value - total_aa_invested[i] - costs
        total_value[i] = total_aa_invested[i] + cash_aa[i]

        prev_shares = aa_shares[i]
        prev_cash = cash_aa[i]

    # Assemble the output frame in the documented column order
    out = pd.DataFrame(index=df.index)

    for j, fund in enumerate(fund_list):
        out[f"{fund}_{price_column}"] = price_arr[:, j]
        out[f"{fund}_Target_Weight"] = weight_arr[:, j]
        out[f"{fund}_BA_Shares"] = ba_shares[:, j]
        out[f"{fund}_BA_$_Invested"] = ba_invested[:, j]

    out["Total_BA_$_Invested"] = total_ba_invested
    out["Cash_Interest"] = cash_interest
    out["Cash_BA"] = cash_ba
    out["Contribution"] = cash_contrib
```

```
out["Rebalance"] = np.where(rebal_flags, "Yes", "No")

for j, fund in enumerate(fund_list):
    out[f"{fund}_AA_Shares"] = aa_shares[:, j]
    out[f"{fund}_AA_$_Invested"] = aa_invested[:, j]
    out[f"{fund}_Notional"] = notional[:, j]

out["Total_AA_$_Invested"] = total_aa_invested
out["Total_Transaction_Costs"] = total_costs
out["Cash_AA"] = cash_aa
out["Total_Value"] = total_value

# Realized weights, calculated once total value is known
for j, fund in enumerate(fund_list):
    out[f"{fund}_AA_%_Invested"] = aa_invested[:, j] / total_value

out["Total_AA_%_Invested"] = total_aa_invested / total_value
out["Cash_%"] = cash_aa / total_value

# Whether the portfolio was actually on margin. The two pass cost sizing can
# leave cash a few billionths negative even at leverage 1.0, which is
# rounding, not borrowing, so judge against a tolerance scaled to portfolio
# size rather than against zero.
out["Borrowing"] = out["Cash_AA"] < -1e-6 * out["Total_Value"].abs()

# Performance
out["Return"] = out["Total_Value"].pct_change()
out["Cumulative_Return"] = (1 + out["Return"]).cumprod()
out["Cum_Total_Transaction_Costs"] = out["Total_Transaction_Costs"].cumsum()
out["Cum_Cash_Interest"] = out["Cash_Interest"].cumsum()

plan_name = "_".join(fund_list)

# Export to excel
if excel_export == True:
    out.to_excel(f"{plan_name}_Strategy.xlsx", sheet_name="data")
else:
    pass

# Export to pickle
if pickle_export == True:
    out.to_pickle(f"{plan_name}_Strategy.pkl")
else:
    pass

# Output confirmation
if output_confirmation == True:
    borrowed_rows = out["Borrowing"].sum()
```

```
        print(
            f"Strategy complete for {plan_name} at {leverage}x leverage, "
            f"borrowing on {borrowed_rows} of {num_rows} rows"
        )
    else:
        pass

    return out
```

0.6.6 bb_clean_data

```
import os
import pandas as pd

from IPython.display import display


def bb_clean_data(
    base_directory: str,
    fund_ticker_name: str,
    source: str,
    asset_class: str,
    excel_export: bool,
    pickle_export: bool,
    output_confirmation: bool,
) -> pd.DataFrame:
    """
    This function takes an excel export from Bloomberg and removes all
    excess data leaving date and close columns.

    Parameters
    ----------
    base_directory : str
        Root path to store downloaded data.
    fund_ticker_name : str
        The fund to clean the data from.
    source : str
        Name of the data source (e.g., 'Bloomberg').
    asset_class : str
        Asset class name (e.g., 'Equities').
    excel_export : bool
        If True, export data to Excel format.
    pickle_export : bool
        If True, export data to Pickle format.
    output_confirmation : bool
        If True, print confirmation message.

    Returns
```

```
    -------
    df : pd.DataFrame
        DataFrame containing cleaned data prices.

    Raises
    ------
    FileNotFoundError:
        If the excel export for `fund_ticker_name` has not been downloaded.
    ValueError:
        If the export contains no header row beginning with "Date".
    """

    # Set location from where to read existing excel file
    location = f"{base_directory}/{source}/{asset_class}/Daily/{fund_ticker_name}.xlsx"

    # Read data from excel, treating every row as data so the header row can be
    # located below
    try:
        df = pd.read_excel(
            location, sheet_name="Worksheet", engine="calamine", header=None
        )
    except FileNotFoundError as e:
        raise FileNotFoundError(
            f"File not found...please download the data for {fund_ticker_name}: {location}"
        ) from e

    # Find the header row - Bloomberg writes a variable number of metadata rows
    # (Security, Start Date, End Date, Period, and sometimes Currency or Pricing
    # Source) above the row whose first cell is "Date"
    header_rows = df.index[df.iloc[:, 0].astype(str).str.strip() == "Date"]
    if len(header_rows) == 0:
        raise ValueError(f"No row starting with 'Date' found in {location}")
    header_row = header_rows[0]

    # Set the column headings from the header row
    df.columns = df.iloc[header_row]

    # Set the column heading for the index to be "None"
    df.rename_axis(None, axis=1, inplace=True)

    # Drop the metadata rows along with the header row itself
    df.drop(df.index[0 : header_row + 1], inplace=True)

    # Set the date column as the index
    df.set_index("Date", inplace=True)

    # Drop the volume column
    try:
```

```
        df.drop(columns={"PX_VOLUME"}, inplace=True)
    except KeyError:
        pass

    # Rename column
    try:
        df.rename(columns={"PX_LAST": "Close"}, inplace=True)
        df.rename(columns={"PX_OPEN": "Open"}, inplace=True)
        df.rename(columns={"PX_HIGH": "High"}, inplace=True)
        df.rename(columns={"PX_LOW": "Low"}, inplace=True)
    except KeyError:
        pass

    # Sort by date
    df.sort_values(by=["Date"], inplace=True)

    # Create directory
    directory = f"{base_directory}/{source}/{asset_class}/Daily"
    os.makedirs(directory, exist_ok=True)

    # Export to excel
    if excel_export == True:
        df.to_excel(f"{directory}/{fund_ticker_name}_Clean.xlsx", sheet_name="data")
    else:
        pass

    # Export to pickle
    if pickle_export == True:
        df.to_pickle(f"{directory}/{fund_ticker_name}_Clean.pkl")
    else:
        pass

    # Output confirmation
    if output_confirmation == True:
        print(f"The first and last date of data for {fund_ticker_name} is: ")
        display(df[:1])
        display(df[-1:])
        print(f"Bloomberg data cleaning complete for {fund_ticker_name}")
        print(f"--------------------")
    else:
        pass

    return df
```

0.6.7 build_index

```
from pathlib import Path
```

```python
def build_index() -> None:
    """
    Build a Hugo-compatible index.md by combining Markdown fragments.

    This function reads a template file (`index_temp.md`) and a list of markdown
    dependencies from `index_dep.txt`. For each entry in the dependency list, it
    replaces a corresponding placeholder in the template
    (formatted as <!-- INSERT_<name>_HERE -->) with the content from the markdown
    file. If a file is missing, the placeholder is replaced with a warning note.

    Output
    ------
    - Writes the final assembled content to `index.md`.

    Raises
    ------
    FileNotFoundError:
        If either `index_temp.md` or `index_dep.txt` does not exist.

    Example
    -------
    If `index_dep.txt` contains:
        01_intro.md
        02_analysis.md

    And `index_temp.md` contains:
        <!-- INSERT_01_intro_HERE -->
        <!-- INSERT_02_analysis_HERE -->

    The resulting `index.md` will include the contents of the respective markdown files in pla
    of their placeholders.
    """

    temp_index_path = Path("index_temp.md")
    final_index_path = Path("index.md")
    dependencies_path = Path("index_dep.txt")

    # Read the index template
    if not temp_index_path.exists():
        raise FileNotFoundError("Missing index_temp.md")

    temp_index_content = temp_index_path.read_text()

    # Read dependency list
    if not dependencies_path.exists():
        raise FileNotFoundError("Missing index_dep.txt")
```

```
    with dependencies_path.open("r") as f:
        markdown_files = [line.strip() for line in f if line.strip()]

    # Replace placeholders for each dependency
    final_index_content = temp_index_content
    for md_file in markdown_files:
        placeholder = f"<!-- INSERT_{Path(md_file).stem}_HERE -->"
        if Path(md_file).exists():
            content = Path(md_file).read_text()
            final_index_content = final_index_content.replace(placeholder, content)
        else:
            print(
                f"  Warning: {md_file} not found, skipping placeholder {placeholder}"
            )
            final_index_content = final_index_content.replace(
                placeholder, f"*{md_file} not found*"
            )

    # Write final index.md
    final_index_path.write_text(final_index_content)
    print("  index.md successfully built!")


if __name__ == "__main__":
    build_index()
```

0.6.8 calc_fed_cycle_asset_performance

```
import numpy as np
import pandas as pd


def calc_fed_cycle_asset_performance(
    start_date: pd.Series,
    end_date: pd.Series,
    label: pd.Series,
    fed_funds_change: pd.Series,
    monthly_returns: pd.DataFrame,
) -> pd.DataFrame:

    results = []

    for start, end, label, fed_change in zip(
        start_date, end_date, label, fed_funds_change
    ):
        start = pd.to_datetime(start)
        end = pd.to_datetime(end)
```

```
    # Filter returns for the cycle period
    returns = monthly_returns.loc[start:end, "Monthly_Return"]

    if len(returns) == 0:
        continue

    cumulative_return = (1 + returns).prod() - 1
    average_return = returns.mean()
    volatility = returns.std()

    results.append(
        {
            "Cycle": label,
            "Start": start.date(),
            "End": end.date(),
            "Months": len(returns),
            "CumulativeReturn": cumulative_return,
            "AverageMonthlyReturn": average_return,
            "Volatility": volatility,
        }
    )

# Convert to DataFrame
cycle_df = pd.DataFrame(results)
cycle_df["CumulativeReturnPct"] = 100 * cycle_df["CumulativeReturn"]
cycle_df["AverageMonthlyReturnPct"] = 100 * cycle_df["AverageMonthlyReturn"]
cycle_df["AnnualizedReturn"] = (1 + cycle_df["CumulativeReturn"]) ** (
    12 / cycle_df["Months"]
) - 1
cycle_df["AnnualizedReturnPct"] = 100 * cycle_df["AnnualizedReturn"]

# Correct the volatility calculation to annualized volatility
cycle_df["Volatility"] = cycle_df["Volatility"] * np.sqrt(12)

# Re-order columns
cycle_df = cycle_df[
    [
        "Cycle",
        "Start",
        "End",
        "Months",
        "CumulativeReturn",
        "CumulativeReturnPct",
        "AverageMonthlyReturn",
        "AverageMonthlyReturnPct",
        "AnnualizedReturn",
        "AnnualizedReturnPct",
        "Volatility",
```

```
        ]
    ]

    # Merge Fed changes into cycle_df
    cycle_df["FedFundsChange"] = fed_funds_change
    cycle_df["FedFundsChange_bps"] = (
        cycle_df["FedFundsChange"] * 10000
    )  # in basis points

    # Add annualized change in FFR in basis points
    cycle_df["FFR_AnnualizedChange"] = (
        cycle_df["FedFundsChange"] / cycle_df["Months"]
    ) * 12
    cycle_df["FFR_AnnualizedChange_bps"] = (
        cycle_df["FFR_AnnualizedChange"] * 10000
    )  # Convert to basis points

    cycle_df["Label"] = cycle_df.apply(
        lambda row: f"{row['Cycle']}, {row['Start']} to {row['End']}", axis=1
    )

    return cycle_df
```

0.6.9 calc_money_weighted_return

```
import pandas as pd

from scipy.optimize import brentq


def calc_money_weighted_return(
    contributions: pd.Series,
    terminal_value: float,
    terminal_date: pd.Timestamp,
) -> float:
    """
    Calculate the annualized money weighted return (IRR) of a series of deposits.

    Finds the annual rate at which every deposit, compounded from its own date to
    ``terminal_date`` on a 365 day basis, grows to exactly ``terminal_value``.

    Parameters
    ----------
    contributions : pd.Series
        Date-indexed Series of dollars deposited. Zero entries are ignored.
    terminal_value : float
        Portfolio value at ``terminal_date``, including any uninvested cash.
    terminal_date : pd.Timestamp
```

```
    Date the portfolio is valued.

Returns
-------
float
    Annualized money weighted return in decimal form, e.g. 0.08 for 8%.

Raises
------
ValueError
    If there are no deposits, or no rate between -99% and 1000% matches
    ``terminal_value``.

Example
-------
>>> deposits = pd.Series([100.0], index=[pd.Timestamp("2020-01-01")])
>>> calc_money_weighted_return(deposits, 110.0, pd.Timestamp("2020-12-31"))
0.1002...
"""

deposits = contributions[contributions != 0]
if deposits.empty:
    raise ValueError("  contributions contains no deposits.")

years = (terminal_date - deposits.index).days.to_numpy(dtype=float) / 365
amounts = deposits.to_numpy(dtype=float)

def valuation_error(rate: float) -> float:
    return (amounts * (1 + rate) ** years).sum() - terminal_value

if valuation_error(-0.99) * valuation_error(10.0) > 0:
    raise ValueError(
        f"  No rate between -99% and 1000% matches terminal value {terminal_value}."
    )

return brentq(valuation_error, -0.99, 10.0, xtol=1e-10)
```

0.6.10 calc_vix_trade_pnl

```
import pandas as pd


def calc_vix_trade_pnl(
    transaction_df: pd.DataFrame,
    exp_start_date: str | None,
    exp_end_date: str | None,
    trade_start_date: str | None,
    trade_end_date: str | None,
```

```
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, str, str, str, str]:
    """
    Calculate the profit and loss (PnL) of trades based on transaction data.

    Parameters
    ----------
    transaction_df : pd.DataFrame
        DataFrame containing transaction data.
    exp_start_date : str | None
        Start date for filtering transactions in 'YYYY-MM-DD' format. This is the start of the
    exp_end_date : str | None
        End date for filtering transactions in 'YYYY-MM-DD' format. This is the end of the rang
    trade_start_date : str | None
        Start date for filtering transactions in 'YYYY-MM-DD' format. This is the start of the
    trade_end_date : str | None
        End date for filtering transactions in 'YYYY-MM-DD' format. This is the end of the rang

    Returns
    -------
    transactions_data : pd.DataFrame
        Dataframe containing the transactions for the specified timeframe.
    closed_trades : pd.DataFrame
        DataFrame containing the closed trades with realized PnL and percent PnL.
    open_trades : pd.DataFrame
        DataFrame containing the open trades.
    net_PnL_percent_str : str
        String representation of the net profit percentage.
    net_PnL_str : str
        String representation of the net profit and loss in dollars.
    """

    # If start and end dates for trades and expirations are None, use the entire DataFrame
    if (
        exp_start_date is None
        and exp_end_date is None
        and trade_start_date is None
        and trade_end_date is None
    ):
        transactions_data = transaction_df

    # If both start and end dates for trades and expirations are provided then filter by both
    else:
        transactions_data = transaction_df[
            (transaction_df["Exp_Date"] >= exp_start_date)
            & (transaction_df["Exp_Date"] <= exp_end_date)
            & (transaction_df["Trade_Date"] >= trade_start_date)
            & (transaction_df["Trade_Date"] <= trade_end_date)
        ]
```

```
# Combine the 'Action' and 'Symbol' columns to create a unique identifier for each transac
transactions_data["TradeDate_Action_Symbol_VIX"] = (
    transactions_data["Trade_Date"].astype(str)
    + ", "
    + transactions_data["Action"]
    + ", "
    + transactions_data["Symbol"]
    + ", VIX = "
    + transactions_data["Approx_VIX_Level"].astype(str)
)

# Split buys and sells and sum the notional amounts
transactions_sells = transactions_data[
    transactions_data["Action"] == "Sell to Close"
]
transactions_sells = transactions_sells.groupby(
    ["Symbol", "Exp_Date"], as_index=False
)[["Amount", "Quantity"]].sum()

transactions_buys = transactions_data[transactions_data["Action"] == "Buy to Open"]
transactions_buys = transactions_buys.groupby(
    ["Symbol", "Exp_Date"], as_index=False
)[["Amount", "Quantity"]].sum()

# Merge buys and sells dataframes back together
merged_transactions = pd.merge(
    transactions_buys,
    transactions_sells,
    on=["Symbol", "Exp_Date"],
    how="outer",
    suffixes=("_Buy", "_Sell"),
)
merged_transactions = merged_transactions.sort_values(
    by=["Exp_Date"], ascending=[True]
)
merged_transactions = merged_transactions.reset_index(drop=True)

# Identify the closed positions
merged_transactions["Closed"] = (
    (~merged_transactions["Amount_Sell"].isna())
    & (~merged_transactions["Amount_Buy"].isna())
    & (merged_transactions["Quantity_Buy"] == merged_transactions["Quantity_Sell"])
)

# Create a new dataframe for closed positions
closed_trades = merged_transactions[merged_transactions["Closed"]]
closed_trades = closed_trades.reset_index(drop=True)
```

```
closed_trades["Realized_PnL"] = (
    closed_trades["Amount_Sell"] - closed_trades["Amount_Buy"]
)
closed_trades["Percent_PnL"] = (
    closed_trades["Realized_PnL"] / closed_trades["Amount_Buy"]
)
closed_trades.drop(columns={"Closed", "Exp_Date"}, inplace=True)
closed_trades["Quantity_Sell"] = closed_trades["Quantity_Sell"].astype(int)

# Calculate the net % PnL
net_PnL_percent = (
    closed_trades["Realized_PnL"].sum() / closed_trades["Amount_Buy"].sum()
)
net_PnL_percent_str = f"{round(net_PnL_percent * 100, 2)}%"

# Calculate the net $ PnL
net_PnL = closed_trades["Realized_PnL"].sum()
net_PnL_str = f"${net_PnL:,.2f}"

# Create a new dataframe for open positions
open_trades = merged_transactions[~merged_transactions["Closed"]]
open_trades = open_trades.reset_index(drop=True)
open_trades.drop(
    columns={"Closed", "Amount_Sell", "Quantity_Sell", "Exp_Date"}, inplace=True
)

# Calculate the total market value of opened positions
# If start and end dates for trades and expirations are None, use only the closed position
if (
    exp_start_date is None
    and exp_end_date is None
    and trade_start_date is None
    and trade_end_date is None
):
    total_opened_pos_mkt_val = closed_trades["Amount_Buy"].sum()
else:
    total_opened_pos_mkt_val = (
        closed_trades["Amount_Buy"].sum() + open_trades["Amount_Buy"].sum()
    )
total_opened_pos_mkt_val_str = f"${total_opened_pos_mkt_val:,.2f}"

# Calculate the total market value of closed positions
total_closed_pos_mkt_val = closed_trades["Amount_Sell"].sum()
total_closed_pos_mkt_val_str = f"${total_closed_pos_mkt_val:,.2f}"

return (
    transactions_data,
    closed_trades,
```

```
        open_trades,
        net_PnL_percent_str,
        net_PnL_str,
        total_opened_pos_mkt_val_str,
        total_closed_pos_mkt_val_str,
    )
```

0.6.11 calculate_rsi

```
import pandas as pd


def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculates the Relative Strength Index (RSI) for a price series."""
    delta = prices.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)

    avg_gain = gain.ewm(alpha=1 / period, adjust=False).mean()
    avg_loss = loss.ewm(alpha=1 / period, adjust=False).mean()

    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))
    return rsi
```

0.6.12 coinbase_fetch_available_products

```
import pandas as pd
import requests


def coinbase_fetch_available_products(
    base_currency: str,
    quote_currency: str,
    status: str,
) -> pd.DataFrame:
    """
    Fetch available products from Coinbase Exchange API.

    Parameters:
    -----------
    base_currency : str, optional
        Filter products by base currency (e.g., 'BTC').
    quote_currency : str, optional
        Filter products by quote currency (e.g., 'USD').
    status : str, optional
        Filter products by status (e.g., 'online', 'offline').
```

```
    Returns:
    --------
    pd.DataFrame
        DataFrame containing available products with their details.
    """

    url = "https://api.exchange.coinbase.com/products"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        products = response.json()

        # Convert the list of products into a pandas DataFrame
        df = pd.DataFrame(products)

        # Filter by base_currency if provided
        if base_currency:
            df = df[df["base_currency"] == base_currency]

        # Filter by quote_currency if provided
        if quote_currency:
            df = df[df["quote_currency"] == quote_currency]

        # Filter by status if provided
        if status:
            df = df[df["status"] == status]

        # Sort by "id"
        df = df.sort_values(by="id")

        return df

    except requests.exceptions.HTTPError as errh:
        print(f"HTTP Error: {errh}")
    except requests.exceptions.ConnectionError as errc:
        print(f"Error Connecting: {errc}")
    except requests.exceptions.Timeout as errt:
        print(f"Timeout Error: {errt}")
    except requests.exceptions.RequestException as err:
        print(f"Oops: Something Else {err}")


if __name__ == "__main__":

    # Example usage
    df = coinbase_fetch_available_products(
        base_currency=None,
        quote_currency="USD",
```

```
        status="online",
    )

    if df is not None:
        print(df)
    else:
        print("No data returned.")
```

0.6.13 coinbase_fetch_full_history

```
import pandas as pd
import time

from coinbase_fetch_historical_candles import coinbase_fetch_historical_candles
from datetime import datetime, timedelta


def coinbase_fetch_full_history(
    product_id: str,
    start: datetime,
    end: datetime,
    granularity: int,
) -> pd.DataFrame:
    """
    Fetch full historical data for a given product from Coinbase Exchange API.

    Parameters:
    -----------
    product_id : str
        The trading pair (e.g., 'BTC-USD').
    start : datetime
        Start time in UTC.
    end : datetime
        End time in UTC.
    granularity : int
        Time slice in seconds (e.g., 3600 for hourly candles).

    Returns:
    --------
    pd.DataFrame
        DataFrame containing time, low, high, open, close, volume.
    """

    all_data = []
    current_start = start
    window = timedelta(seconds=granularity * 300)  # Fetch max 300 candles per request

    while current_start < end:
```

```
            current_end = min(current_start + window, end)
            df = coinbase_fetch_historical_candles(
                product_id, current_start, current_end, granularity
            )

            if df.empty:
                # No candles in this window. Step over it rather than stopping: a
                # gap in the data longer than the window would otherwise truncate
                # the history at that point. This matters most at minute
                # granularity, where a window spans only 5 hours
                current_start = current_end
            else:
                all_data.append(df)
                current_start = df["time"].iloc[-1] + timedelta(seconds=granularity)

            time.sleep(0.2)  # Small delay to respect rate limits

        if all_data:
            full_df = pd.concat(all_data).reset_index(drop=True)
            return full_df
        else:
            return pd.DataFrame()


if __name__ == "__main__":

    # Example usage
    df = coinbase_fetch_full_history(
        product_id="BTC-USD",
        start=datetime(2025, 1, 1),
        end=datetime(2025, 1, 31),
        granularity=86_400,
    )

    if df is not None:
        print(df)
    else:
        print("No data returned.")
```

0.6.14 coinbase_fetch_historical_candles

```
import pandas as pd
import requests
import time

from datetime import datetime
```

```python
def coinbase_fetch_historical_candles(
    product_id: str,
    start: datetime,
    end: datetime,
    granularity: int,
) -> pd.DataFrame:
    """
    Fetch historical candle data for a given product from Coinbase Exchange API.

    Parameters:
    -----------
    product_id : str
        The trading pair (e.g., 'BTC-USD').
    start : str
        Start time in UTC.
    end : str
        End time in UTC.
    granularity : int
        Time slice in seconds (e.g., 60 for minute candles, 3600 for hourly candles, 86,400 fo

    Returns:
    --------
    pd.DataFrame
        DataFrame containing time, low, high, open, close, volume.
    """

    url = f"https://api.exchange.coinbase.com/products/{product_id}/candles"
    params = {
        "start": start.isoformat(),
        "end": end.isoformat(),
        "granularity": granularity,
    }

    max_retries = 5
    retry_delay = 1  # initial delay in seconds

    for attempt in range(max_retries):
        try:
            response = requests.get(url, params=params, timeout=10)
            response.raise_for_status()
            data = response.json()

            # Coinbase Exchange API returns data in reverse chronological order
            data = data[::-1]

            # Convert to DataFrame
            df = pd.DataFrame(
                data, columns=["time", "low", "high", "open", "close", "volume"]
```

```
            )
            df["time"] = pd.to_datetime(df["time"], unit="s")
            return df

        except requests.exceptions.HTTPError as errh:
            if response.status_code == 429:
                print(f"Rate limit exceeded. Retrying in {retry_delay} seconds...")
                time.sleep(retry_delay)
                retry_delay *= 2  # Exponential backoff
            else:
                print(f"HTTP Error: {errh}")
                break
        except requests.exceptions.ConnectionError as errc:
            print(f"Error Connecting: {errc}")
            time.sleep(retry_delay)
            retry_delay *= 2
        except requests.exceptions.Timeout as errt:
            print(f"Timeout Error: {errt}")
            time.sleep(retry_delay)
            retry_delay *= 2
        except requests.exceptions.RequestException as err:
            print(f"OOps: Something Else {err}")
            break

    raise Exception("Failed to fetch data after multiple retries.")


if __name__ == "__main__":

    # Example usage
    df = coinbase_fetch_historical_candles(
        product_id="BTC-USD",
        start=datetime(2025, 1, 1),
        end=datetime(2025, 1, 2),
        granularity=3_600,
    )

    if df is not None:
        print(df)
    else:
        print("No data returned.")
```

0.6.15 coinbase_pull_data

```
import os
import pandas as pd

from coinbase_fetch_available_products import coinbase_fetch_available_products
```

```
from coinbase_fetch_full_history import coinbase_fetch_full_history
from datetime import datetime, timedelta
from IPython.display import display
from settings import config

# Get the data directory from the configuration
DATA_DIR = config("DATA_DIR")

# Supported granularities, in seconds, mapped to the subdirectory the data is
# stored in
GRANULARITY_DIRECTORIES = {
    60: "Minute",
    3600: "Hourly",
    86400: "Daily",
}


def _find_first_available_date(
    product_id: str,
    start_date: datetime,
    end_date: datetime,
) -> datetime:
    """
    Locate the first date with data for a product using daily candles.

    Pulling at daily granularity covers 300 days per request, so scanning a
    range of years costs only a handful of requests. Scanning the same range at
    minute granularity, where each request covers 5 hours, would take thousands.

    Parameters:
    -----------
    product_id : str
        The trading pair (e.g., 'BTC-USD').
    start_date : datetime
        Earliest date to consider.
    end_date : datetime
        Latest date to consider.

    Returns:
    --------
    datetime
        Date of the first available candle, or None if the product has no data.
    """

    daily_history_df = coinbase_fetch_full_history(
        product_id, start_date, end_date, 86400
    )
```

```
    if daily_history_df.empty:
        return None

    return daily_history_df["time"].min()


def coinbase_pull_data(
    base_directory,
    product_id: str,
    source: str,
    asset_class: str,
    granularity: int = 3600,  # 60=minute, 3600=hourly, 86400=daily
    start_date: datetime = datetime(2010, 1, 1),
    end_date: datetime = None,  # defaults to 1 day ago due to lag in data availability
    force_existing_check: bool = False,
    excel_export: bool = False,
    pickle_export: bool = True,
    parquet_export: bool = False,
    output_confirmation: bool = True,
    verbose: bool = False,
) -> pd.DataFrame:
    """
    Read existing data file, download price data from Coinbase, and export data.

    Parameters:
    -----------
    base_directory : any
        Root path to store downloaded data.
    product_id : str
        The trading pair to download (e.g., 'BTC-USD').
    source : str
        Name of the data source (e.g., 'Coinbase').
    asset_class : str
        Asset class name (e.g., 'Cryptocurrencies').
    granularity : int
        Time slice in seconds (60=minute, 3600=hourly, 86400=daily).
    start_date : datetime
        Earliest date to consider. Only used when no existing record is found;
        where a record exists it is resumed from its last stored candle.
    end_date : datetime, optional
        End date in UTC. Defaults to 1 day ago due to lag in data availability.
    force_existing_check : bool
        If True, re-pull from start_date rather than resuming, to fill any gaps
        in an existing data file.
    excel_export : bool
        If True, export data to Excel format.
    pickle_export : bool
        If True, export data to Pickle format.
```

```
    parquet_export : bool
        If True, export data to Parquet format.
    output_confirmation : bool
        If True, print confirmation message.
    verbose : bool
        If True, print detailed information about the data being processed.

    Returns:
    --------
    pd.DataFrame
        DataFrame containing the updated price data, indexed by Date.
    """

    # Resolved here rather than in the signature so that the default tracks the
    # current date instead of the moment this module was imported
    if end_date is None:
        end_date = datetime.now() - timedelta(days=1)

    if granularity not in GRANULARITY_DIRECTORIES:
        raise ValueError(
            f"Invalid granularity: {granularity}. Acceptable values are: {sorted(GRANULARITY_DI
        )

    time_length = GRANULARITY_DIRECTORIES[granularity]

    # Set file location based on parameters
    directory = f"{base_directory}/{source}/{asset_class}/{time_length}"
    file_location = f"{directory}/{product_id}.pkl"

    try:
        # Attempt to read existing pickle data file
        existing_history_df = pd.read_pickle(file_location)

        # Reset index if 'Date' is the index
        if "Date" not in existing_history_df.columns:
            existing_history_df = existing_history_df.reset_index()

        print(f"File found...updating the {product_id} {time_length} data.")

        if verbose == True:
            print("Existing data:")
            print(existing_history_df)

        # Find last date in existing data
        last_data_date = existing_history_df["Date"].max()
        starting_rows = len(existing_history_df)
        print(f"Last date in existing data: {last_data_date}")
        print(f"Number of rows in existing data: {starting_rows}")
```

```
        # Resume from the last stored candle. It is re-fetched because the
        # period may still have been in progress when it was written
        current_start = last_data_date

    except FileNotFoundError:
        # Print error
        print(f"File not found...downloading the {product_id} {time_length} data.")

        # Create an empty DataFrame so that both paths converge on the same
        # fetch and merge below
        existing_history_df = pd.DataFrame(
            {
                "Date": pd.Series(dtype="datetime64[ns]"),
                "low": pd.Series(dtype="float64"),
                "high": pd.Series(dtype="float64"),
                "open": pd.Series(dtype="float64"),
                "close": pd.Series(dtype="float64"),
                "volume": pd.Series(dtype="float64"),
            }
        )

        # Set the number of starting rows to be 0
        starting_rows = 0

        # Find where the product's data actually begins rather than walking
        # forward from start_date at the requested granularity
        first_available_date = _find_first_available_date(
            product_id, start_date, end_date
        )

        if first_available_date is None:
            print(f"No data available for {product_id}.")
            return existing_history_df.set_index("Date")

        print(f"First available candle for {product_id}: {first_available_date}")
        current_start = first_available_date

    # Check for force_existing_check flag
    if force_existing_check == True:
        print("Forcing check of existing data...")
        current_start = start_date

    # Pull the data
    new_data = coinbase_fetch_full_history(
        product_id, current_start, end_date, granularity
    )
```

```
if new_data.empty:
    # Nothing returned means the record is already current
    print(f"No new data returned...{product_id} {time_length} data is up to date.")
    full_history_df = existing_history_df.copy()

else:
    new_data = new_data.rename(columns={"time": "Date"})

    if verbose == True:
        print("New data:")
        print(new_data)

    # Combine existing data with recent data, keeping the newly fetched row
    # where a date appears in both so that a partial candle is replaced by
    # its completed version
    full_history_df = pd.concat([existing_history_df, new_data])
    full_history_df = full_history_df.drop_duplicates(subset="Date", keep="last")
    full_history_df = full_history_df.sort_values(by="Date", ascending=True)
    full_history_df = full_history_df.reset_index(drop=True)

    if verbose == True:
        print("Combined data:")
        print(full_history_df)

# Store with Date as the index, matching the existing data files
full_history_df = full_history_df.set_index("Date")

# Create directory
os.makedirs(directory, exist_ok=True)

# Export to Excel
if excel_export == True:
    print(f"Exporting {product_id} {time_length} data to Excel...")
    full_history_df.to_excel(f"{directory}/{product_id}.xlsx", sheet_name="data")

# Export to Pickle
if pickle_export == True:
    print(f"Exporting {product_id} {time_length} data to Pickle...")
    full_history_df.to_pickle(f"{directory}/{product_id}.pkl")

# Export to Parquet
if parquet_export == True:
    print(f"Exporting {product_id} {time_length} data to Parquet...")
    full_history_df.to_parquet(f"{directory}/{product_id}.parquet")

total_rows = len(full_history_df)

# Output confirmation
```

```
    if output_confirmation == True:
        print(f"The first and last date of {time_length} data for {product_id} is: ")
        display(full_history_df[:1])
        display(full_history_df[-1:])
        print(f"Number of rows after data update: {total_rows}")

        if starting_rows:
            print(f"Number of rows added during update: {total_rows - starting_rows}")

        print(f"Coinbase data complete for {product_id} {time_length} data.")
        print(f"--------------------")

    return full_history_df


if __name__ == "__main__":

    # Stablecoins and wrapped coins are not useful for analysis
    excluded_products = [
        "DAI-USD",
        "GUSD-USD",
        "PAX-USD",
        "PYUSD-USD",
        "USDC-USD",
        "USDT-USD",
        "WAXL-USD",
        "WBTC-USD",
    ]

    # Identify the products to pull
    available_products = coinbase_fetch_available_products(
        base_currency=None,
        quote_currency="USD",
        status="online",
    )
    product_list = sorted(
        product
        for product in available_products["id"]
        if product not in excluded_products
    )

    # Limit to a subset of currencies for this example
    currencies = ["BTC", "ETH", "SOL", "XRP"]
    product_list = [
        product for product in product_list if product.split("-")[0] in currencies
    ]

    # Timespans
```

```
    granularities = [60, 3600, 86400]

    # Iterate through each product
    for product in product_list:
        for granularity in granularities:
            try:
                coinbase_pull_data(
                    base_directory=DATA_DIR,
                    product_id=product,
                    source="Coinbase",
                    asset_class="Cryptocurrencies",
                    granularity=granularity,
                    force_existing_check=False,
                    excel_export=granularity != 60,  # minute data is pickle only
                    pickle_export=True,
                    parquet_export=False,
                    output_confirmation=True,
                    verbose=False,
                )
            except Exception as e:
                print(f"Failed to pull {product} at granularity {granularity}: {e}")
```

0.6.16 compute_daily_performance

```
import pandas as pd


def compute_daily_performance(
    tickers: list,
    data: pd.DataFrame,
    trades: pd.DataFrame,
    initial_capital: int,
) -> pd.DataFrame:
    """
    Computes daily portfolio equity, return, etc. from trades dataframe.

    Parameters:
    -----------
    tickers : list
        List of crypto tickers, e.g., ["BTC-USD", "ETH-USD", "SOL-USD"].
    data : pd.DataFrame
        DataFrame containing merged price data for all tickers.
    trades : pd.DataFrame
        DataFrame of trades from backtest_rsi_multi_asset_strategy.
    initial_capital : int
        Initial capital for the portfolio.

    Returns:
```

```
    --------
    pd.DataFrame
        DataFrame containing daily data for the portfolio with equity, cash,
        returns, positions, drawdowns, and prices.
    """

    # Copy the data dataframe
    df = data.copy().set_index("Date")

    # Filter the entry trades
    entry_trades_df = trades.copy()
    entry_trades_df = entry_trades_df[
        ["asset", "entry_time", "entry_price", "quantity", "entry_fee"]
    ]
    entry_trades_df["cash"] = (
        entry_trades_df["entry_price"] * entry_trades_df["quantity"] * -1
    ) - entry_trades_df["entry_fee"]
    entry_trades_df["crypto_value"] = (
        entry_trades_df["quantity"] * entry_trades_df["entry_price"]
    )
    entry_trades_df.rename(columns={"entry_time": "Date"}, inplace=True)

    # Filter the exit trades
    exit_trades_df = trades.copy()
    exit_trades_df = exit_trades_df[
        ["asset", "exit_time", "exit_price", "quantity", "exit_fee"]
    ]
    exit_trades_df["cash"] = (
        exit_trades_df["exit_price"] * exit_trades_df["quantity"]
    ) - exit_trades_df["exit_fee"]
    exit_trades_df["quantity"] = (
        exit_trades_df["quantity"] * -1
    )  # Convert quantity to negative for exit
    exit_trades_df["crypto_value"] = (
        exit_trades_df["quantity"] * exit_trades_df["exit_price"]
    )
    exit_trades_df.rename(columns={"exit_time": "Date"}, inplace=True)

    # Combine entry and exit trades into a single DataFrame
    ledger_events_df = pd.concat([entry_trades_df, exit_trades_df], ignore_index=True)
    ledger_events_df = ledger_events_df.sort_values("Date")

    # Create pivot table for quantity held by date per asset
    quantity_df = ledger_events_df.pivot_table(
        index="Date",
        columns="asset",
        values="quantity",
        aggfunc="sum",
```

```
).fillna(0)

quantity_df.columns = [f"{col}_qty" for col in quantity_df.columns]

# Cash flow dataframe
cash_df = ledger_events_df.groupby("Date")["cash"].sum().to_frame()

# Combine quantity and cash dataframes into the ledger
ledger_qtys_df = pd.concat([quantity_df, cash_df], axis=1).fillna(0)
ledger_qtys_df = ledger_qtys_df.sort_index()

# Isolate the daily close prices for each asset
price_cols = [f"{ticker}_close" for ticker in tickers]
price_df = df[price_cols].copy()

# Merge the daily close prices DataFrame into the ledger
ledger_qtys_prices_df = ledger_qtys_df.join(price_df, how="outer")

# Replace all NaN values with 0
ledger_qtys_prices_df.fillna(0, inplace=True)

# Add columns for each asset's cumulative quantity
quantity_cols = [
    col for col in ledger_qtys_prices_df.columns if col.endswith("_qty")
]
for col in quantity_cols:
    ledger_qtys_prices_df[col] = ledger_qtys_prices_df[col].cumsum()

# Cumulative cash column
ledger_qtys_prices_df["cash"] = ledger_qtys_prices_df["cash"].cumsum()

# Add initial capital amount to cash to represent initial capital
ledger_qtys_prices_df["cash"] += initial_capital

# Establish position columns for each asset
ledger_qtys_prices_pos_df = ledger_qtys_prices_df.copy()
for col in quantity_cols:
    asset_symbol = col.replace("_qty", "")
    ledger_qtys_prices_pos_df[f"{asset_symbol}_position"] = (
        ledger_qtys_prices_pos_df[col]
        * ledger_qtys_prices_pos_df[f"{asset_symbol}_close"]
    )

# Re-arrange the columns to have date, cash, then quantities, prices, positions grouped by
asset_symbols = []
for col in quantity_cols:
    asset_symbol = col.replace("_qty", "")
    asset_symbols.append(asset_symbol)
```

```
cols = ["cash"] + [
    col
    for ticker in asset_symbols
    for col in ledger_qtys_prices_pos_df.columns
    if col.startswith(ticker)
]
ledger_qtys_prices_pos_df = ledger_qtys_prices_pos_df[cols]

# Calculate total portfolio value
ledger_qtys_prices_pos_df["equity"] = ledger_qtys_prices_pos_df[
    "cash"
] + ledger_qtys_prices_pos_df[
    [col for col in ledger_qtys_prices_pos_df.columns if col.endswith("_position")]
].sum(
    axis=1
)

# Reindex ledger to daily using last price for the day
daily_ledger_qtys_prices_pos_df = ledger_qtys_prices_pos_df.copy()
daily_ledger_qtys_prices_pos_df = daily_ledger_qtys_prices_pos_df.resample(
    "D"
).last()

# Drop the columns where any of the crypto asset prices = 0
price_cols = [
    col for col in daily_ledger_qtys_prices_pos_df.columns if col.endswith("_close")
]
complete_ledger = daily_ledger_qtys_prices_pos_df[
    ~(daily_ledger_qtys_prices_pos_df[price_cols] == 0).any(axis=1)
]

# Calculate crypto daily return, cumulative return, and drawdown
for col in asset_symbols:
    complete_ledger[f"{col}_return"] = complete_ledger[f"{col}_close"].pct_change()
    complete_ledger[f"{col}_cum_return"] = (
        1 + complete_ledger[f"{col}_return"]
    ).cumprod() - 1
    complete_ledger[f"{col}_drawdown"] = (
        complete_ledger[f"{col}_close"] - complete_ledger[f"{col}_close"].cummax()
    ) / complete_ledger[f"{col}_close"].cummax()

# Calculate portfolio daily return and cumulative return
complete_ledger["Return"] = complete_ledger["equity"].pct_change()
complete_ledger["Cum_Return"] = (1 + complete_ledger["Return"]).cumprod() - 1

# Calculate drawdown
complete_ledger["Drawdown"] = (
```

```
        complete_ledger["equity"] - complete_ledger["equity"].cummax()
    ) / complete_ledger["equity"].cummax()

    # Fill NaN values with 0 to return a complete DataFrame
    complete_ledger.fillna(0, inplace=True)

    return complete_ledger
```

0.6.17 create_ma_trend_signals

```
import pandas as pd


def create_ma_trend_signals(
    fund_list: list[str],
    price_df: pd.DataFrame,
    ma_list: list[int],
    ma_type: str,
    price_column: str,
    signal_lag: int,
    output_confirmation: bool,
    fund_weights: list[float] | None = None,
) -> pd.DataFrame:
    """
    Generate multi-asset trend following target weights from a grid of moving averages.

    For every fund, a moving average is calculated for each span in ``ma_list``.
    Each average casts one vote: the vote is 1 when the fund's price is above that
    average and 0 otherwise. The fraction of votes in favor becomes the fund's
    signal strength, a value between 0 (price below every average) and 1 (price
    above every average).

    Signal strength is then scaled by the fund's share of ``fund_weights``, taken
    across the funds carrying valid price data on that row, so that a fully
    invested portfolio sums to a target weight of 1 and the unallocated remainder
    is held in cash. By default every fund carries equal weight and so tops out at
    1/N; passing ``[2, 1, 1]`` for three funds lets the first reach 0.5 and the
    other two 0.25. Funds without price data on a given row receive a target
    weight of 0 and are left out of the normalization, which lets the portfolio
    work with funds whose histories begin at different dates.

    Target weights are finally shifted forward by ``signal_lag`` rows so that a
    portfolio acting on this frame only ever trades on information that was
    already observable. With the default of 1, the weight applied on day t is the
    one implied by the close of day t-1.

    Notes
    -----
```

```
    A fund is only given a signal on rows where every span in the grid has a
    value. A simple moving average needs its full window before it reports
    anything, so a fund holds a target weight of 0 for the first ``max(ma_list)``
    rows of its own history. An exponential moving average is seeded on the first
    observation and reports immediately, which means its early values still carry
    the memory of that seed; allowing roughly one full span of warm-up before
    trading an EMA grid is prudent, but the caller must arrange that by choosing a
    later start date, since the EMA itself gives no signal that it is unready.

    Parameters
    ----------
    fund_list : list[str]
        List of funds to generate signals for, e.g. ``["IVV", "EFA", "EEM"]``.
    price_df : pd.DataFrame
        Date-indexed DataFrame containing a price column per fund, named
        ``f"{fund}_{price_column}"``.
    ma_list : list[int]
        List of moving average spans in rows (trading days for daily data), e.g.
        ``[20, 40, 60]``. Longer lists produce a smoother allocation.
    ma_type : str
        Type of moving average to use, either ``"EMA"`` for exponential or
        ``"SMA"`` for simple. Case insensitive. An EMA weights recent prices more
        heavily and turns faster; an SMA weights every observation in its window
        equally and turns more slowly.
    price_column : str
        Suffix of the price column to build signals from, e.g. ``"Adj_Close"``.
    signal_lag : int
        Number of rows to shift the target weights forward. Use 1 for daily data
        to trade on the prior close's signal. Use 0 only to inspect the
        unlagged signal; a backtest built on it will look ahead.
    output_confirmation : bool
        If True, print confirmation message.
    fund_weights : list[float] | None, optional
        Relative allocation ceiling per fund, in the same order as ``fund_list``.
        Values are normalized, so ``[2, 1, 1]`` and ``[0.5, 0.25, 0.25]`` describe
        the same portfolio. Because the normalization is taken over the funds
        active on each row, a fund's ceiling rises while its peers are still
        outside their price history. A weight of 0 holds a fund out of the
        portfolio while leaving its moving averages and signal strength in the
        returned frame. Defaults to None, which weights every fund equally.

    Returns
    -------
    pd.DataFrame
        Date-indexed DataFrame containing, for each fund, the moving average
        columns (``f"{fund}_{ma_type}_{span}"``), the signal strength
        (``f"{fund}_Signal_Strength"``), and the lagged target weight
        (``f"{fund}_Target_Weight"``), plus ``Total_Target_Weight`` and
```

```
    ``Cash_Weight``.

Raises
------
ValueError
    If ``ma_type`` is not "EMA" or "SMA", ``ma_list`` is empty, ``signal_lag``
    is negative, a required price column is missing from ``price_df``, or
    ``fund_weights`` is the wrong length, negative, non-finite, or all zero.
"""

ma_type = ma_type.strip().upper()

if ma_type not in ("EMA", "SMA"):
    raise ValueError(f" ma_type must be 'EMA' or 'SMA', got {ma_type}.")

if not ma_list:
    raise ValueError(" ma_list must contain at least one moving average span.")

if signal_lag < 0:
    raise ValueError(f" signal_lag must be >= 0, got {signal_lag}.")

missing = [
    f"{fund}_{price_column}"
    for fund in fund_list
    if f"{fund}_{price_column}" not in price_df.columns
]
if missing:
    raise ValueError(f" price_df is missing required columns: {missing}")

if fund_weights is None:
    weights = pd.Series(1.0, index=fund_list, dtype=float)
else:
    if len(fund_weights) != len(fund_list):
        raise ValueError(
            f" fund_weights must have one entry per fund, got "
            f"{len(fund_weights)} weights for {len(fund_list)} funds."
        )

    weights = pd.Series(
        [float(weight) for weight in fund_weights], index=fund_list, dtype=float
    )

    if weights.isna().any() or (weights == float("inf")).any():
        raise ValueError(f" fund_weights must be finite, got {fund_weights}.")

    if (weights < 0).any():
        raise ValueError(
            f" fund_weights must be non-negative, got {fund_weights}."
```

```
        )

    if weights.sum() <= 0:
        raise ValueError(
            "  fund_weights must contain at least one positive weight."
        )

df = price_df.copy()

# Total weight carried by the funds with valid price data on each row. Taking
# the sum per row rather than once up front lets a fund's ceiling grow while
# its peers are still outside their price history. Rows where no fund has data
# are held as NaN so the division below does not blow up to infinity.
active_funds = df[[f"{fund}_{price_column}" for fund in fund_list]].notna()
active_funds.columns = fund_list
active_weight = active_funds.mul(weights, axis=1).sum(axis=1)
active_weight = active_weight.where(active_weight > 0)

for fund in fund_list:
    price = df[f"{fund}_{price_column}"]

    # Calculate an average per span and record whether price sits above it
    votes = []
    for span in ma_list:
        if ma_type == "EMA":
            ma = price.ewm(span=int(span), adjust=False).mean()
        else:
            ma = price.rolling(window=int(span), min_periods=int(span)).mean()

        df[f"{fund}_{ma_type}_{span}"] = ma

        # A missing price or an average that has not filled its window yet
        # is recorded as no opinion rather than as a vote against
        votes.append((price > ma).astype(float).where(price.notna() & ma.notna()))

    # Signal strength is the fraction of averages the price is trading above.
    # skipna=False so a partially warmed up grid yields no signal at all,
    # rather than silently reporting on whichever spans happen to be ready.
    signal_strength = pd.concat(votes, axis=1).mean(axis=1, skipna=False)
    df[f"{fund}_Signal_Strength"] = signal_strength

    # Split the allocation across the funds that are active on that row in
    # proportion to their weight, then lag so the weight is only acted on
    # once it was observable
    target_weight = (signal_strength * weights[fund] / active_weight).fillna(0)
    df[f"{fund}_Target_Weight"] = target_weight.shift(signal_lag).fillna(0)

# Portfolio level weights
```

```
    df["Total_Target_Weight"] = df[[f"{fund}_Target_Weight" for fund in fund_list]].sum(
        axis=1
    )
    df["Cash_Weight"] = 1 - df["Total_Target_Weight"]

    # Output confirmation
    if output_confirmation == True:
        plan_name = "_".join(fund_list)
        print(
            f"Trend signals complete for {plan_name} "
            f"across {len(ma_list)} {ma_type} spans with a {signal_lag} row lag"
        )
    else:
        pass

    return df
```

0.6.18 create_signals

```
import pandas as pd


def create_signals(
    tickers: list,
    data: pd.DataFrame,
    # --- RSI settings ---
    use_rsi: bool,
    rsi_threshold: int,
    # --- MA settings ---
    use_ma: bool,
    ma_days: list,
    # --- Bollinger settings ---
    use_bbands: bool,
    bb_rule: str = "touch_lower",  # {"touch_lower", "cross_up_from_below", "below_lower"}
) -> pd.DataFrame:
    """
    Generate entry signals combining RSI, optional MA filter, and optional Bollinger Bands.

    Notes:
    - Bollinger bands are computed on *_close_prev so the signal at time t only uses info up t
    - BB rules:
        * "touch_lower": close_prev <= lower_prev  (default)
        * "below_lower": close_prev  < lower_prev  (strict)
        * "cross_up_from_below": close_prev crosses from < lower_prev to >= lower_prev (uses p
    """

    # Copy the data dataframe
    df = data.copy()
```

```
signals = []
for ticker in tickers:
    # --- RSI mask ---
    if use_rsi:
        rsi_mask = (
            df.get(f"{ticker}_RSI_prev", pd.Series(index=df.index, dtype="float64"))
            .lt(rsi_threshold)
            .fillna(False)
        )
    else:
        rsi_mask = pd.Series(True, index=df.index, dtype="bool")

    # --- MA mask ---
    if use_ma:
        if ma_days:
            ma_cols = [
                f"{ticker}_MA_{day}d_prev"
                for day in ma_days
                if f"{ticker}_MA_{day}d_prev" in df.columns
            ]
            if not ma_cols:
                # MA periods requested but none present → this ticker yields no signals
                continue

            ma_pass_df = pd.concat(
                [(df[f"{ticker}_close"] > df[col]).rename(col) for col in ma_cols],
                axis=1,
            ).fillna(False)

            ma_passes = ma_pass_df.sum(axis=1)
            allocation_pct = ma_passes / len(ma_days)
            ma_mask = ma_passes.ge(1)
        else:
            ma_passes = pd.Series(0, index=df.index, dtype="int64")
            allocation_pct = pd.Series(
                1.0, index=df.index, dtype="float64"
            )  # full allocation if no MAs
            ma_mask = pd.Series(True, index=df.index, dtype="bool")

    # --- Bollinger Bands mask ---
    if use_bbands:
        # close_prev = df.get(f"{ticker}_close_prev")
        # if close_prev is None:
        #     # If we don't have *_close_prev, skip BB logic safely
        #     bb_mask = pd.Series(True, index=df.index, dtype="bool")  # don't block RSI/M
        #     bb_mid_prev = bb_up_prev = bb_low_prev = pd.Series(pd.NA, index=df.index)
        #     bb_z_prev = pd.Series(pd.NA, index=df.index)
```

```
        # else:
        #     rolling = close_prev.rolling(window=bb_window, min_periods=bb_window)
        #     mid_prev = rolling.mean()
        #     std_prev = rolling.std(ddof=0)
        #     up_prev = mid_prev + bb_num_std * std_prev
        #     low_prev = mid_prev - bb_num_std * std_prev

        #     # Band position (z-score) using prev values
        #     bb_z_prev = (close_prev - mid_prev) / std_prev

        close_prev = df.get(f"{ticker}_close_prev")
        low_prev = df.get(f"{ticker}_BB_LOWER_prev")
        mid_prev = df.get(f"{ticker}_BB_MID_prev")
        up_prev = df.get(f"{ticker}_BB_UPPER_prev")
        bb_z_prev = df.get(f"{ticker}_BB_Z_prev")

        # Choose BB rule
        if bb_rule == "below_lower":
            bb_mask = close_prev.lt(low_prev)
        elif bb_rule == "cross_up_from_below":
            prev_below = close_prev.shift(1).lt(low_prev.shift(1))
            now_at_or_above = close_prev.ge(low_prev)
            bb_mask = prev_below & now_at_or_above
        else:  # "touch_lower"
            bb_mask = close_prev.le(low_prev)

        bb_mask = bb_mask.fillna(False)
        bb_mid_prev, bb_up_prev, bb_low_prev = mid_prev, up_prev, low_prev
    else:
        # BB disabled
        bb_mask = pd.Series(True, index=df.index, dtype="bool")
        bb_mid_prev = bb_up_prev = bb_low_prev = pd.Series(pd.NA, index=df.index)
        bb_z_prev = pd.Series(pd.NA, index=df.index)

    # --- Combined entry rule ---
    mask = rsi_mask & ma_mask & bb_mask
    if not mask.any():
        continue

    # Collect signal rows
    cols = [
        "Date",
        f"{ticker}_open",
        f"{ticker}_high",
        f"{ticker}_low",
        f"{ticker}_close",
        f"{ticker}_close_prev",
    ]
```

```
        # pick only columns that exist (avoids KeyError if some are missing)
        available_cols = [c for c in cols if c in df.columns]
        s = df.loc[mask, available_cols].copy()

        # Add metadata
        s["asset"] = ticker
        s["ma_passes"] = ma_passes.loc[mask].astype(int)
        s["allocation_pct"] = allocation_pct.loc[mask]
        s["bb_rule"] = bb_rule if use_bbands else pd.NA
        s["bb_mid_prev"] = bb_mid_prev.loc[mask]
        s["bb_up_prev"] = bb_up_prev.loc[mask]
        s["bb_low_prev"] = bb_low_prev.loc[mask]
        s["bb_z_prev"] = bb_z_prev.loc[
            mask
        ]  # negative means below mid; ~-2 at/below lower band

        # Standardize column names
        rename_map = {
            f"{ticker}_open": "open",
            f"{ticker}_high": "high",
            f"{ticker}_low": "low",
            f"{ticker}_close": "close",
            f"{ticker}_close_prev": "close_prev",
        }
        s.rename(
            columns={k: v for k, v in rename_map.items() if k in s.columns},
            inplace=True,
        )

        signals.append(s)

    signals_df = (
        pd.concat(signals, ignore_index=True)
        if signals
        else pd.DataFrame(
            columns=[
                "Date",
                "open",
                "high",
                "low",
                "close",
                "asset",
                "ma_passes",
                "allocation_pct",
                "bb_rule",
                "bb_mid_prev",
                "bb_up_prev",
```

```
                "bb_low_prev",
                "bb_z_prev",
                "close_prev",
            ]
        )
    )

    # Sort and de-dup (1 per asset per timestamp)
    if not signals_df.empty:
        signals_df = signals_df.sort_values(by="Date")
        signals_df = signals_df.drop_duplicates(subset=["Date", "asset"])

    return signals_df
```

0.6.19 df_info

```
import pandas as pd
from IPython.display import display


def df_info(
    df: pd.DataFrame,
) -> None:
    """
    Display summary information about a pandas DataFrame.

    This function prints:
    - The DataFrame's column names, shape, and data types via `df.info()`
    - The first 5 rows using `df.head()`
    - The last 5 rows using `df.tail()`

    It uses `display()` for better output formatting in environments like Jupyter notebooks.

    Parameters:
    -----------
    df : pd.DataFrame
        The DataFrame to inspect.

    Returns:
    --------
    None

    Example:
    --------
    >>> df_info(my_dataframe)
    """

    print("The columns, shape, and data types are:")
```

```
    print(df.info())
    print("The first 5 rows are:")
    display(df.head())
    print("The last 5 rows are:")
    display(df.tail())
```

0.6.20 df_info_markdown

```
import io
import pandas as pd


def df_info_markdown(
    df: pd.DataFrame,
    decimal_places: int = 2,
) -> str:
    """
    Generate a Markdown-formatted summary of a pandas DataFrame.

    This function captures and formats the output of `df.info()`, `df.head()`,
    and `df.tail()` in Markdown for easy inclusion in reports, documentation,
    or web-based rendering (e.g., Hugo or Jupyter export workflows).

    Parameters:
    -----------
    df : pd.DataFrame
        The DataFrame to summarize.

    Returns:
    --------
    str
        A string containing the DataFrame's info, head, and tail
        formatted in Markdown.

    Example:
    --------
    >>> print(df_info_markdown(df))
    ```text
    The columns, shape, and data types are:
    <output from df.info()>
    ```
    The first 5 rows are:
	col1	col2
0	...	...

    The last 5 rows are:
    ...
```

```
    """

    buffer = io.StringIO()

    # Capture df.info() output
    df.info(buf=buffer)
    info_str = buffer.getvalue()

    # Convert head and tail to Markdown
    head_str = df.head().to_markdown(floatfmt=f".{decimal_places}f")
    tail_str = df.tail().to_markdown(floatfmt=f".{decimal_places}f")

    markdown = [
        "```text",
        "The columns, shape, and data types are:\n",
        info_str,
        "```",
        "\nThe first 5 rows are:\n",
        # "The first 5 rows are:\n",
        head_str,
        "\nThe last 5 rows are:\n",
        tail_str,
    ]

    # markdown = [
    #     "```text",
    #     "The columns, shape, and data types are:\n",
    #     info_str,
    #     "\nThe first 5 rows are:\n",
    #     head_str,
    #     "\nThe last 5 rows are:\n",
    #     tail_str,
    #     "```"
    # ]

    # markdown = [
    #     "The columns, shape, and data types are:\n",
    #     info_str,
    #     # "\nThe first 5 rows are:\n",
    #     "The first 5 rows are:\n",
    #     head_str,
    #     "\nThe last 5 rows are:\n",
    #     tail_str
    # ]

    return "\n".join(markdown)
```

0.6.21 export_track_md_deps

```
from pathlib import Path


def export_track_md_deps(
    dep_file: Path,
    md_filename: str,
    content: str,
    output_type: str = "markdown",
) -> None:
    """
    Export Markdown content to a file and track it as a dependency.

    This function writes the provided content to the specified
    Markdown file and appends the filename to the given dependency
    file (typically `index_dep.txt`). This is useful in workflows
    where Markdown fragments are later assembled into a larger
    document (e.g., a Hugo `index.md`).

    Parameters
    ----------
    dep_file : Path
        Path to the dependency file that tracks Markdown fragment
        filenames.
    md_filename : str
        The name of the Markdown file to export.
    content : str
        The Markdown-formatted content to write to the file.
    output_type : str, optional
        Indicates whether the content is plain text, Python code,
        or markdown for proper formatting (default: "text").

    Returns
    -------
    None
    """

    if output_type == "python":
        Path(md_filename).write_text(f"```python\n{content}\n```")
    elif output_type == "text":
        Path(md_filename).write_text(f"```text\n{content}\n```")
    elif output_type == "markdown":
        Path(md_filename).write_text(content)
    else:
        raise ValueError(
            "'output_type' must be either 'text', 'python', or 'markdown'."
        )
```

```
    with dep_file.open("a") as f:
        f.write(md_filename + "\n")
    print(f"  Exported and tracked: {md_filename}")
```

0.6.22 filter_top_percent

```
from typing import Iterable, Mapping, Optional, Tuple

import pandas as pd


def filter_top_percent(
    df: pd.DataFrame,
    cols: Iterable[str],
    top_pct: float,
    how: str = "all",
    ascending: Optional[Mapping[str, bool]] = None,
    coerce_numeric: bool = True,
) -> Tuple[pd.DataFrame, pd.Series]:
    """
    Filter rows to the top X% for a set of columns.

    Parameters
    ----------
    df : pd.DataFrame
        Source data. Not modified.
    cols : Iterable[str]
        Columns to evaluate.
    top_pct : float
        Percentage (e.g., 10 or 0.10 for top 10%).
    how : {"all", "any"}, default "all"
        "all": keep rows that are in the top X% for *every* column (intersection).
        "any": keep rows that are in the top X% for *at least one* column (union).
    ascending : dict[str, bool] | None
        Per-column direction. For each col: True => smaller is better (use lower tail),
        False => larger is better (use upper tail). If None, defaults to False for all.
        Example: {"Max Drawdown": True, "MAR Ratio": False}
    coerce_numeric : bool
        If True, converts the specified cols to numeric (non-numeric values become NaN).

    Returns
    -------
    filtered_df : pd.DataFrame
        Rows meeting the criterion.
    thresholds : pd.Series
        Threshold value used for each column (the cutoff included in the keep).
    """
```

```
if 0 < top_pct <= 1:
    frac = top_pct
elif 1 < top_pct <= 100:
    frac = top_pct / 100.0
else:
    raise ValueError("top_pct must be in (0,1] or (1,100].")

# Work on a copy so the caller's DataFrame is left untouched
df = df.copy()

cols = list(cols)
if coerce_numeric:
    for c in cols:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce")

# Any unspecified column defaults to larger-is-better
ascending = {**{c: False for c in cols}, **(ascending or {})}

thresholds = {}
masks = {}

for c in cols:
    if c not in df.columns:
        raise KeyError(f"Column '{c}' not found in DataFrame.")

    # If smaller is better, top X% = bottom X% (lower tail). Else upper tail.
    q = frac if ascending[c] else (1 - frac)

    # Guard against all-NaN columns
    cutoff = df[c].quantile(q) if df[c].notna().any() else float("nan")
    thresholds[c] = cutoff

    # Use quantile cutoff; include ties on the boundary
    if ascending[c]:
        masks[c] = df[c] <= cutoff
    else:
        masks[c] = df[c] >= cutoff

thresholds = pd.Series(thresholds)

if how == "all":
    keep_mask = pd.concat(masks, axis=1).all(axis=1)
elif how == "any":
    keep_mask = pd.concat(masks, axis=1).any(axis=1)
else:
    raise ValueError("how must be 'all' or 'any'.")
```

```
    filtered_df = df[keep_mask].copy()

    return filtered_df, thresholds
```

0.6.23 load_crypto_data

```
import pandas as pd

from load_data import load_data


def load_crypto_data(
    tickers: list,
    base_directory,
    start_date: str,
    end_date: str,
) -> pd.DataFrame:
    """
    Loads minute-level data for multiple crypto tickers from Coinbase source.


    Parameters:
    -----------
    tickers : list
        List of crypto tickers, e.g., ["BTC-USD", "ETH-USD", "SOL-USD"].
    base_directory
        Base directory where data files are stored.
    start_date : str
        Optional start date for filtering data, e.g., "2023-01-01".
    end_date : str
        Optional end date for filtering data, e.g., "2023-12-31".

    Returns:
    --------
    pd.DataFrame
        DataFrame containing merged price data for all tickers.
    """

    # Create empty DataFrame
    df = pd.DataFrame()

    for ticker in tickers:
        # Load data
        temp_df = load_data(
            base_directory=base_directory,
            ticker=ticker,
            source="Coinbase",
```

```
            asset_class="Cryptocurrencies",
            timeframe="Minute",
            file_format="pickle",
        )
        temp_df.index = pd.to_datetime(temp_df.index)
        temp_df = temp_df.sort_index()

        # Rename columns
        temp_df = temp_df.rename(
            columns={
                "open": f"{ticker}_open",
                "high": f"{ticker}_high",
                "low": f"{ticker}_low",
                "close": f"{ticker}_close",
                "volume": f"{ticker}_volume",
            }
        )

        # Merge the data
        df = pd.merge(df, temp_df, how="outer", left_index=True, right_index=True)

    # Apply date filtering if specified
    if start_date:
        df = df[df.index >= pd.to_datetime(start_date)]
    if end_date:
        df = df[df.index <= pd.to_datetime(end_date)]

    # Reset index
    df = df.reset_index()

    return df
```

0.6.24 load_data

```
import pandas as pd
from pathlib import Path


def load_data(
    base_directory,
    ticker: str,
    source: str,
    asset_class: str,
    timeframe: str,
    file_format: str,
) -> pd.DataFrame:
    """
    Load data from a CSV, Excel, or Pickle file into a pandas DataFrame.
```

```
    This function attempts to read a file first as a CSV, then as an Excel file
    (specifically looking for a sheet named 'data' and using the 'calamine' engine).
    If both attempts fail, a ValueError is raised.

    Parameters:
    -----------
    base_directory
        Root path to read data file.
    ticker : str
        Ticker symbol to read.
    source : str
        Name of the data source (e.g., 'Yahoo').
    asset_class : str
        Asset class name (e.g., 'Equities').
    timeframe : str
        Timeframe for the data (e.g., 'Daily', 'Month_End').
    file_format : str
        Format of the file to load ('csv', 'excel', or 'pickle')

    Returns:
    --------
    pd.DataFrame
        The loaded data.

    Raises:
    -------
    ValueError
        If the file could not be loaded as either CSV or Excel.

    Example:
    --------
    >>> df = load_data(DATA_DIR, "^VIX", "Yahoo_Finance", "Indices")
    """

    if file_format == "csv":
        csv_path = (
            Path(base_directory) / source / asset_class / timeframe / f"{ticker}.csv"
        )
        df = pd.read_csv(csv_path)
        return df

    elif file_format == "excel":
        xlsx_path = (
            Path(base_directory) / source / asset_class / timeframe / f"{ticker}.xlsx"
        )
        df = pd.read_excel(xlsx_path, sheet_name="data", engine="calamine")
        return df
```

```
    elif file_format == "pickle":
        pickle_path = (
            Path(base_directory) / source / asset_class / timeframe / f"{ticker}.pkl"
        )
        df = pd.read_pickle(pickle_path)
        return df

    else:
        raise ValueError(
            f" Unsupported file format: {file_format}. Please use 'csv', 'excel', or 'pickle'.
        )
```

0.6.25 pandas_set_decimal_places

```
import pandas as pd


def pandas_set_decimal_places(
    decimal_places: int,
) -> None:
    """
    Set the number of decimal places displayed for floating-point numbers in pandas.

    Parameters
    ----------
    decimal_places : int
        The number of decimal places to display for float values in pandas DataFrames and Seri

    Returns
    -------
    None

    Example
    -------
    >>> dp(3)
    >>> pd.DataFrame([1.23456789])
           0
    0   1.235
    """

    pd.set_option("display.float_format", lambda x: f"%.{decimal_places}f" % x)
```

0.6.26 plot_bar_returns_ffr_change

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

```python
def plot_bar_returns_ffr_change(
    cycle_df: pd.DataFrame,
    asset_label: str,
    annualized_or_cumulative: str,
    caption: str = None,
) -> None:

    plt.figure(figsize=(10, 8))

    if annualized_or_cumulative == "Cumulative":

        # Create bar plot for cumulative returns
        barplot = sns.barplot(
            data=cycle_df, x="Label", y="CumulativeReturnPct", palette="coolwarm"
        )

        # Annotate each bar with cumulative return and Fed rate change
        for i, row in cycle_df.iterrows():
            barplot.text(
                i,
                row["CumulativeReturnPct"] + 1,
                f"{row['CumulativeReturnPct']:.1f}%\nΔFFR: {row['FedFundsChange_bps']:.0f}bps"
                color="black",
                ha="center",
                # fontsize=10,
            )

    elif annualized_or_cumulative == "Annualized":

        # Create bar plot for annualized returns
        barplot = sns.barplot(
            data=cycle_df,
            x="Label",  # Date range labels from earlier
            y="AnnualizedReturnPct",
            palette="coolwarm",
        )

        # Annotate each bar with annualized return + annualized FFR change
        for i, row in cycle_df.iterrows():
            barplot.text(
                i,
                row["AnnualizedReturnPct"] + 1,
                f"{row['AnnualizedReturnPct']:.1f}%\nΔFFR: {row['FFR_AnnualizedChange_bps']:.0
                color="black",
                ha="center",
                # fontsize=10,
```

```
        )

    # plt.ylabel(f"{asset_label} {annualized_or_cumulative} Return (%)", fontsize=14)
    plt.ylabel(f"{asset_label} {annualized_or_cumulative} Return (%)")
    # plt.yticks(fontsize=12)
    plt.yticks()
    # plt.xlabel("Fed Policy Cycle (Date Range)", fontsize=14)
    plt.xlabel("Fed Policy Cycle (Date Range)")
    # plt.xticks(rotation=45, ha="right", fontsize=12)
    plt.xticks(rotation=45, ha="right")
    # plt.title(f"{asset_label} {annualized_or_cumulative} Return by Fed Policy Cycle With {an
    plt.title(
        f"{asset_label} {annualized_or_cumulative} Return by Fed Policy Cycle With {annualized_
    )
    plt.tight_layout()

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)

    plt.show()
```

0.6.27 plot_histogram

```
import math
import matplotlib.pyplot as plt
import pandas as pd

from matplotlib.ticker import MultipleLocator

from round_to_nice_value import round_to_nice_value


def plot_histogram(
    df: pd.DataFrame,
    plot_columns: str | list[str],
    title: str,
    x_label: str,
    x_tick_spacing: int | float | str,
    x_tick_rotation: int,
    y_label: str,
    y_tick_spacing: int | float | str,
    y_tick_rotation: int,
    grid: bool = True,
    legend: bool = True,
    legend_location: str = "best",
    legend_anchor: tuple = None,
    export_plot: bool = False,
```

```
    plot_file_name: str = None,
    caption: str = None,
) -> None:
    """
    Plot a histogram of one or more columns from a DataFrame, with lines marking
    the mean and +/- 1 and 2 standard deviations of each.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame containing the data to plot.
    plot_columns : str OR list
        Column or list of columns to plot from the DataFrame, or "All" to plot
        every column.
    title : str
        Title of the plot.
    x_label : str
        Label for the x-axis.
    x_tick_spacing : int OR float OR str
        Spacing for the x-axis ticks, or "Auto" to derive it from the range of
        the histogram bins across all plotted columns.
    x_tick_rotation : int
        Rotation angle for the x-axis tick labels.
    y_label : str
        Label for the y-axis.
    y_tick_spacing : int OR float OR str
        Spacing for the y-axis ticks, or "Auto" to derive it from the tallest
        bar across all plotted columns (at least 1, since counts are whole
        numbers).
    y_tick_rotation : int
        Rotation angle for the y-axis tick labels.
    grid : bool, optional
        Whether to display a grid on the plot (default is True).
    legend : bool, optional
        Whether to display a legend on the plot (default is True).
    legend_location : str, optional
        Location of the legend on the plot (default is "best").
    legend_anchor : tuple, optional
        Anchor point (x, y) for placing the legend relative to the axes,
        e.g. (1, 1) to move it outside the plot (default is None).
    export_plot : bool, optional
        Whether to save the figure as a PNG file (default is False).
    plot_file_name : str, optional
        File name for saving the figure (if export_plot is True, default is None).
    caption : str, optional
        Caption drawn centered below the figure, e.g. "Figure 1" (default is
        None).
```

```
    Returns
    -------
    None
    """

    # Set plot figure size and background color
    plt.figure(figsize=(10, 6))

    # Plot data
    cols = df.columns if plot_columns == "All" else plot_columns

    # Track the extent across all columns so "Auto" spacing fits every histogram
    max_count = 0
    bins_min = math.inf
    bins_max = -math.inf

    for col in cols:
        mean = df[col].mean()
        std = df[col].std()
        # Create histogram first to get its color
        n, bins, patches = plt.hist(
            df[col], label=col, bins=200, edgecolor="black", alpha=0.5
        )
        max_count = max(max_count, n.max())
        bins_min = min(bins_min, bins[0])
        bins_max = max(bins_max, bins[-1])

        hist_color = patches[0].get_facecolor()
        # Use histogram color for vertical lines
        plt.axvline(
            mean,
            color=hist_color,
            linestyle="dashed",
            linewidth=1,
            label=f"Mean: {mean:.3f}",
        )
        plt.axvline(
            mean + std,
            color=hist_color,
            linestyle="dashed",
            linewidth=1,
            label=f"Mean + 1 std: {mean + std:.3f}",
        )
        plt.axvline(
            mean - std,
            color=hist_color,
            linestyle="dashed",
            linewidth=1,
```

```
            label=f"Mean - 1 std: {mean - std:.3f}",
        )
        plt.axvline(
            mean + 2 * std,
            color=hist_color,
            linestyle="dashed",
            linewidth=1,
            label=f"Mean + 2 std: {mean + 2 * std:.3f}",
        )
        plt.axvline(
            mean - 2 * std,
            color=hist_color,
            linestyle="dashed",
            linewidth=1,
            label=f"Mean - 2 std: {mean - 2 * std:.3f}",
        )

    # Format X axis
    if x_tick_spacing == "Auto":
        raw_x_spacing = (bins_max - bins_min) / 10
        x_tick_spacing = round_to_nice_value(raw_x_spacing)

    plt.gca().xaxis.set_major_locator(MultipleLocator(x_tick_spacing))
    plt.xlabel(x_label)

    if x_tick_rotation != 0:
        # Line up the x-ticks with the labels when rotated
        plt.xticks(rotation=x_tick_rotation, ha="right")
    else:
        plt.xticks(rotation=x_tick_rotation)

    # Format Y axis
    if y_tick_spacing == "Auto":
        # Bars are drawn from 0, and counts are whole numbers
        raw_y_spacing = max_count / 10
        y_tick_spacing = max(1, round_to_nice_value(raw_y_spacing))

    plt.gca().yaxis.set_major_locator(MultipleLocator(y_tick_spacing))
    plt.ylabel(y_label)
    plt.yticks(rotation=y_tick_rotation)

    # Format title, layout, grid, and legend
    plt.title(title)
    plt.tight_layout()

    # Grid
    if grid == True:
        plt.grid(True, linestyle="--", alpha=0.5)
```

```
    # Legend
    if legend == True:
        if legend_anchor is not None:
            plt.legend(loc=legend_location, bbox_to_anchor=legend_anchor)
        else:
            plt.legend(loc=legend_location)

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)

    # Save figure and display plot
    if export_plot == True:
        plt.savefig(f"{plot_file_name}.png", dpi=300, bbox_inches="tight")

    # Display the plot
    plt.show()

    return None
```

0.6.28 plot_iteration_results

```
import matplotlib.pyplot as plt
import pandas as pd


def plot_iteration_results(
    results_df: pd.DataFrame,
) -> None:
    """
    Plot the distribution of backtest parameter sweep results.

    Draws histograms of the MAR ratio and of each swept parameter, followed by
    scatter plots of the MAR ratio against each parameter and against the other
    performance statistics.

    Parameters
    ----------
    results_df : pd.DataFrame
        DataFrame of backtest results, one row per parameter combination. Must
        contain the columns MAR Ratio, MA_DAYS, RSI_PERIOD, RSI_THRESHOLD,
        TRAILING_STOP_PCT, Total Return, Win Rate, Max Drawdown, Total Trades,
        CAGR, Annualized Mean Return, Annualized Volatility, and Annualized
        Sharpe Ratio.

    Returns
    -------
```

```
    None
    """

    df = results_df.copy()

    # Histograms: (column, title, x-axis label)
    histograms = [
        ("MAR Ratio", "Distribution of MAR Ratio", "MAR Ratio"),
        ("MA_DAYS", "Distribution of MA Days", "MA Days"),
        ("RSI_PERIOD", "Distribution of RSI Period", "RSI Period"),
        ("RSI_THRESHOLD", "Distribution of RSI Threshold", "RSI Threshold"),
        (
            "TRAILING_STOP_PCT",
            "Distribution of Trailing Stop Percentage",
            "Trailing Stop Percentage",
        ),
    ]

    for col, title, xlabel in histograms:
        plt.figure(figsize=(12, 6))
        plt.hist(df[col], bins=200, density=True, alpha=0.6, color="g")
        plt.title(title)
        plt.xlabel(xlabel)
        plt.ylabel("Frequency")
        plt.grid()
        plt.show()

    # Scatter plots of MAR Ratio vs: (column, x-axis label)
    scatters = [
        ("MA_DAYS", "MA_DAYS"),
        ("RSI_PERIOD", "RSI_PERIOD"),
        ("RSI_THRESHOLD", "RSI_THRESHOLD"),
        ("TRAILING_STOP_PCT", "TRAILING_STOP"),
        ("Total Return", "Total Return"),
        ("Win Rate", "Win Rate"),
        ("Max Drawdown", "Max Drawdown"),
        ("Total Trades", "Total Trades"),
        ("CAGR", "CAGR"),
        ("Annualized Mean Return", "Annualized Mean Return"),
        ("Annualized Volatility", "Annualized Volatility"),
        ("Annualized Sharpe Ratio", "Annualized Sharpe Ratio"),
    ]

    for col, xlabel in scatters:
        plt.figure(figsize=(12, 6))
        plt.scatter(df[col], df["MAR Ratio"], alpha=0.6)
        plt.title(f"MAR Ratio vs. {xlabel}")
        plt.xlabel(xlabel)
```

```
        plt.ylabel("MAR Ratio")
        plt.grid()
        plt.show()

    return None
```

0.6.29 plot_multi_asset_equity_and_drawdown

```
import matplotlib.pyplot as plt
import pandas as pd

from label_start_end_min_max import label_start_end_min_max
from matplotlib.ticker import StrMethodFormatter
from pathlib import Path


def plot_multi_asset_equity_and_drawdown(
    tickers: list[str],
    daily_perf: pd.DataFrame,
    trades: pd.DataFrame,
    data: pd.DataFrame,
    title: str,
    show_plot: bool,
    export_plot: bool,
    export_dir: Path | str = None,  # <-- new optional parameter
    caption: str = None,
) -> None:
    """
    Plots:
    - Total equity, cash, and per-asset position values
    - Cumulative returns (strategy and crypto assets)
    - Drawdowns (strategy and crypto assets)
    - Price series and trade markers for BTC, ETH, SOL

    Parameters:
    -----------
    tickers : list
        List of crypto tickers, e.g., ["BTC-USD", "ETH-USD", "SOL-USD"].
    daily_perf : pd.DataFrame
        DataFrame containing daily performance metrics from compute_daily_performance.
    trades : pd.DataFrame
        DataFrame of trades from backtest_rsi_multi_asset_strategy.
    data : pd.DataFrame
        DataFrame containing merged price data for all tickers.
    title : str
        Title for the overall plot.
    show_plot : bool
        Whether to display the plot.
```

```
    export_plot : bool
        Whether to save the plot as a PNG file.
    export_dir : Path | str, optional
        Directory to save the plot if export_plot is True. Defaults to current directory.
    caption : str, optional
        Caption drawn centered below the figure, e.g. "Figure 1". Defaults to None.

    Returns:
    --------
    None
    """

    # Copy the data dataframe
    prices_df = data.copy().set_index("Date")

    # Drop NaN rows
    perf_df = daily_perf.copy().dropna()

    # Compute position values and crypto drawdowns
    colors = ["blue", "red", "purple", "orange", "yellow"]

    # Determine number of subplots
    n_fixed = 3  # ax1, ax2, ax3
    n_tickers = len(tickers)
    total_axes = n_fixed + n_tickers

    # Setup plot
    fig, axes = plt.subplots(
        total_axes,
        1,
        figsize=(14, 4 * total_axes),
        sharex=True,
        gridspec_kw={"height_ratios": [1] * total_axes},
    )

    # Unpack fixed axes
    ax1, ax2, ax3 = axes[:3]

    # Assign price axes
    price_axes = axes[3:]

    # ax1: Equity, Cash, and Position Values
    ax1.plot(perf_df.index, perf_df["equity"], color="black", label="Strategy Equity")
    label_start_end_min_max(
        ax1, perf_df.index, perf_df["equity"], fmt="{:,.0f}", fontsize=10
    )
    ax1.plot(
        perf_df.index,
```

```
    perf_df["cash"],
    color="green",
    alpha=0.6,
    linestyle="--",
    label="Strategy Cash",
)
for i, symbol in enumerate(tickers):
    ax1.plot(
        perf_df.index,
        perf_df[f"{symbol}_position"],
        alpha=0.6,
        linestyle="--",
        color=colors[i % len(colors)],
        label=f"{symbol} Strategy Position Value",
    )
ax1.set_title("Equity, Cash, and Position Values (Daily Data)", fontsize=12)
ax1.yaxis.set_major_formatter(StrMethodFormatter("{x:,.0f}"))
ax1.set_ylabel("Account Value ($)")
ax1.legend()
ax1.grid(True)

# ax2: Cumulative Returns
ax2.plot(
    perf_df.index,
    perf_df["Cum_Return"] * 100,
    color="black",
    label="Strategy Return",
)
label_start_end_min_max(
    ax2, perf_df.index, perf_df["Cum_Return"] * 100, fmt="{:.1f}%", fontsize=10
)
for i, symbol in enumerate(tickers):
    ax2.plot(
        perf_df.index,
        perf_df[f"{symbol}_cum_return"] * 100,
        linestyle="--",
        color=colors[i % len(colors)],
        label=f"{symbol} Buy-Hold Return",
    )
    label_start_end_min_max(
        ax2,
        perf_df.index,
        perf_df[f"{symbol}_cum_return"] * 100,
        fmt="{:.1f}%",
        fontsize=10,
    )
ax2.set_title("Cumulative Return (Daily Data)", fontsize=12)
ax2.set_ylabel("Return (%)")
```

```
ax2.legend()
ax2.grid(True)

# ax3: Drawdowns
ax3.plot(
    perf_df.index,
    perf_df["Drawdown"] * 100,
    color="black",
    label="Strategy Drawdown",
)
label_start_end_min_max(
    ax3, perf_df.index, perf_df["Drawdown"] * 100, fmt="{:.1f}%", fontsize=10
)
for i, symbol in enumerate(tickers):
    ax3.plot(
        perf_df.index,
        perf_df[f"{symbol}_drawdown"] * 100,
        linestyle="--",
        color=colors[i % len(colors)],
        label=f"{symbol} Buy-Hold Drawdown",
    )
    label_start_end_min_max(
        ax3,
        perf_df.index,
        y=perf_df[f"{symbol}_drawdown"] * 100,
        fmt="{:.1f}%",
        fontsize=10,
    )
ax3.set_title("Drawdown (Daily Data)", fontsize=12)
ax3.set_ylabel("Drawdown (%)")
ax3.legend()
ax3.grid(True)

# ax4, ax5, ax6... : Price plots with trade markers are dynamically created based on the n
for i, symbol in enumerate(tickers):
    df = prices_df[[f"{symbol}_close"]].copy()
    df.index = pd.to_datetime(df.index)
    # plot_df = df[df.index >= trades["entry_time"].min()]
    plot_df = df.copy()
    ax = price_axes[i]
    ax.plot(
        plot_df.index,
        plot_df[f"{symbol}_close"],
        color="gray",
        label=f"{symbol} Price",
    )
    label_start_end_min_max(
        ax, plot_df.index, plot_df[f"{symbol}_close"], fmt="{:,.0f}", fontsize=10
```

```
)

# Mark trades
asset_trades = trades[trades["asset"] == symbol]
for _, trade in asset_trades.iterrows():
    if trade["quantity"] > 0:
        entry_time = trade["entry_time"]
        exit_time = trade["exit_time"]
        if entry_time in plot_df.index:
            ax.plot(
                entry_time,
                plot_df.loc[entry_time, f"{symbol}_close"],
                marker="^",
                color="green",
                label=(
                    "Entry"
                    if "Entry" not in ax.get_legend_handles_labels()[1]
                    else ""
                ),
            )
        if exit_time in plot_df.index:
            ax.plot(
                exit_time,
                plot_df.loc[exit_time, f"{symbol}_close"],
                marker="v",
                color="red",
                label=(
                    "Exit"
                    if "Exit" not in ax.get_legend_handles_labels()[1]
                    else ""
                ),
            )
    else:
        entry_time = trade["entry_time"]
        exit_time = trade["exit_time"]
        if entry_time in plot_df.index:
            ax.plot(
                entry_time,
                plot_df.loc[entry_time, f"{symbol}_close"],
                marker="^",
                color="lightgreen",
                label=(
                    "Entry (No trade due to position conflict)"
                    if "Entry (No trade due to position conflict)"
                    not in ax.get_legend_handles_labels()[1]
                    else ""
                ),
            )
```

```
            if exit_time in plot_df.index:
                ax.plot(
                    exit_time,
                    plot_df.loc[exit_time, f"{symbol}_close"],
                    marker="v",
                    color="pink",
                    label=(
                        "Exit (No trade due to position conflict)"
                        if "Exit (No trade due to position conflict)"
                        not in ax.get_legend_handles_labels()[1]
                        else ""
                    ),
                )
    ax.set_title(f"{symbol} Price History With Trades (Minute Data)", fontsize=12)
    ax.yaxis.set_major_formatter(StrMethodFormatter("{x:,.0f}"))
    ax.set_ylabel("Price ($)")
    ax.legend()
    ax.grid(True)

# Set global title and layout
fig.suptitle(title, fontsize=16, y=0.99)
plt.tight_layout()

# Caption
if caption is not None:
    fig.text(0.5, 0, caption, ha="center", va="top", fontsize=9)

# Export plot as PNG
if export_plot:
    export_dir = Path(export_dir) if export_dir else Path.cwd()
    export_dir.mkdir(parents=True, exist_ok=True)
    # Expand the saved image to include a caption below the figure
    plt.savefig(
        export_dir / "multi_asset_strategy.png",
        bbox_inches="tight" if caption is not None else None,
    )

# Show the plot
if show_plot == True:
    plt.show()
else:
    plt.close(fig)

return None
```

0.6.30 plot_scatter

```
import math
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

import numpy as np
import pandas as pd

from matplotlib.ticker import FuncFormatter, MultipleLocator, PercentFormatter
from run_regression import run_regression

from round_to_nice_value import round_to_nice_value


def plot_scatter(
    df: pd.DataFrame,
    x_plot_column: str,
    y_plot_columns: list,
    title: str,
    x_label: str,
    x_format: str,
    x_format_decimal_places: int | str,
    x_tick_spacing: int | float | str,
    x_tick_start: str,
    x_tick_rotation: int,
    y_label: str,
    y_format: str,
    y_format_decimal_places: int | str,
    y_tick_spacing: int | float | str,
    y_tick_rotation: int,
    plot_OLS_regression_line: bool = False,
    OLS_column: str = None,
    plot_Ridge_regression_line: bool = False,
    Ridge_column: str = None,
    plot_RidgeCV_regression_line: bool = False,
    RidgeCV_column: str = None,
    regression_constant: bool = True,
    grid: bool = True,
    legend: bool = True,
    legend_location: str = "best",
    legend_anchor: tuple = None,
    export_plot: bool = False,
    plot_file_name: str = None,
    caption: str = None,
) -> None:
    """
    Plot one or more columns of a DataFrame against another column as a scatter
```

```
    plot, with optional regression lines.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame containing the price data to plot.
    x_plot_column : str
        Column to plot on the x-axis from the DataFrame.
    y_plot_columns : list
        Columns to plot on the y-axis from the DataFrame.
    title : str
        Title of the plot.
    x_label : str
        Label for the x-axis.
    x_format : str
        Format for the x-axis labels: "Decimal", "Percentage", "Scientific", or
        "Log" for numeric data; "Second", "Minute", "Hour", "Day", "Week",
        "Month", or "Year" for dates; or "String" for no formatting.
    x_format_decimal_places : int OR str
        Number of decimal places for x-axis labels, or "Auto" to infer it from
        the x-axis tick spacing. Used only by the numeric formats.
    x_tick_spacing : int OR float OR str
        Spacing for the x-axis ticks, or "Auto" to derive it from the range of
        the x-axis data. "Auto" applies to "Decimal", "Percentage", and
        "Scientific" only; date formats need an explicit interval. Ignored
        when x_format is "Log" or "String".
    x_tick_start : str
        Start date for the x-axis ticks.
    x_tick_rotation : int
        Rotation angle for the x-axis tick labels.
    y_label : str
        Label for the y-axis.
    y_format : str
        Format for the y-axis labels: "Decimal", "Percentage", "Scientific", or
        "Log".
    y_format_decimal_places : int OR str
        Number of decimal places for y-axis labels, or "Auto" to infer it from
        the y-axis tick spacing.
    y_tick_spacing : int OR float OR str
        Spacing for the y-axis ticks, or "Auto" to derive it from the range of
        the plotted data. Ignored when y_format is "Log".
    y_tick_rotation : int
        Rotation angle for the y-axis tick labels.
    plot_OLS_regression_line : bool, optional
        Whether to plot an OLS regression line on the scatter plot (default: False).
    OLS_column : str, optional
        Column name for the OLS regression line (default: None).
    plot_Ridge_regression_line : bool, optional
```

```
    Whether to plot a Ridge regression line on the scatter plot (default: False).
Ridge_column : str, optional
    Column name for the Ridge regression line (default: None).
plot_RidgeCV_regression_line : bool, optional
    Whether to plot a RidgeCV regression line on the scatter plot (default: False).
RidgeCV_column : str, optional
    Column name for the RidgeCV regression line (default: None).
regression_constant : bool, optional
    Whether to include a constant term in the regression models (default: True).
grid : bool, optional
    Whether to display a grid on the plot (default is True).
legend : bool, optional
    Whether to display a legend on the plot (default is True).
legend_location : str, optional
    Location of the legend on the plot (default is "best").
legend_anchor : tuple, optional
    Anchor point (x, y) for placing the legend relative to the axes,
    e.g. (1, 1) to move it outside the plot (default is None).
export_plot : bool, optional
    Whether to save the figure as a PNG file (default is False).
plot_file_name : str, optional
    File name for saving the figure (if export_plot is True, default is None).
caption : str, optional
    Caption drawn centered below the figure, e.g. "Figure 1" (default is
    None).

Returns
-------
None
"""

# Set plot figure size and background color
plt.figure(figsize=(10, 6))

# Plot data
for col in y_plot_columns:
    plt.scatter(df[x_plot_column], df[col], label=col, marker="o", alpha=0.5)

# Resolve "Auto" x-tick spacing for numeric x-axes only
# (date-based formats use mdates locators and ignore x_tick_spacing as a numeric value)
numeric_x_formats = ("Decimal", "Percentage", "Scientific", "Log")
if x_tick_spacing == "Auto":
    if x_format in numeric_x_formats:
        raw_x_spacing = (df[x_plot_column].max() - df[x_plot_column].min()) / 20
        x_tick_spacing = round_to_nice_value(raw_x_spacing)
    else:
        # Date-based or string x-axis: "Auto" is not meaningful; pick a safe default
        # so downstream code that may reference x_tick_spacing doesn't fail.
```

```
        x_tick_spacing = 1

if x_format_decimal_places == "Auto":
    if isinstance(x_tick_spacing, float):
        x_tick_spacing_decimal_places = max(
            0, -int(math.floor(math.log10(abs(x_tick_spacing))))
        )
    else:
        x_tick_spacing_decimal_places = 0

    # PercentFormatter shows the value x 100, which needs 2 fewer decimals
    if x_format == "Percentage":
        x_tick_spacing_decimal_places = max(0, x_tick_spacing_decimal_places - 2)

    x_format_decimal_places = x_tick_spacing_decimal_places

# Format X axis
if x_format == "Decimal":
    plt.gca().xaxis.set_major_formatter(
        FuncFormatter(lambda x, _: f"{x:,.{x_format_decimal_places}f}")
    )
elif x_format == "Percentage":
    plt.gca().xaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=x_format_decimal_places)
    )
elif x_format == "Scientific":
    plt.gca().xaxis.set_major_formatter(
        FuncFormatter(lambda x, _: f"{x:.{x_format_decimal_places}e}")
    )
elif x_format == "Log":
    plt.xscale("log")
elif x_format == "Second":
    if x_tick_start is not None:
        ticks = pd.date_range(
            start=pd.Timestamp(x_tick_start).tz_convert(df[x_plot_column].dt.tz),
            end=df[x_plot_column].max() + pd.tseries.offsets.Second(x_tick_spacing),
            freq=pd.tseries.offsets.Second(x_tick_spacing),
        )
        plt.gca().set_xticks(ticks)
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M:%S"))
    else:
        plt.gca().xaxis.set_major_locator(
            mdates.SecondLocator(interval=x_tick_spacing)
        )
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M:%S"))
elif x_format == "Minute":
    if x_tick_start is not None:
        ticks = pd.date_range(
```

```
            start=pd.Timestamp(x_tick_start).tz_convert(df[x_plot_column].dt.tz),
            end=df[x_plot_column].max() + pd.tseries.offsets.Minute(x_tick_spacing),
            freq=pd.tseries.offsets.Minute(x_tick_spacing),
        )
        plt.gca().set_xticks(ticks)
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    else:
        plt.gca().xaxis.set_major_locator(
            mdates.MinuteLocator(interval=x_tick_spacing)
        )
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
elif x_format == "Hour":
    if x_tick_start is not None:
        ticks = pd.date_range(
            start=pd.Timestamp(x_tick_start).tz_convert(df[x_plot_column].dt.tz),
            end=df[x_plot_column].max() + pd.tseries.offsets.Hour(x_tick_spacing),
            freq=pd.tseries.offsets.Hour(x_tick_spacing),
        )
        plt.gca().set_xticks(ticks)
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    else:
        plt.gca().xaxis.set_major_locator(
            mdates.HourLocator(interval=x_tick_spacing)
        )
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
elif x_format == "Day":
    plt.gca().xaxis.set_major_locator(mdates.DayLocator(interval=x_tick_spacing))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%d %b %Y"))
elif x_format == "Week":
    plt.gca().xaxis.set_major_locator(
        mdates.WeekdayLocator(interval=x_tick_spacing)
    )
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%d %b %Y"))
elif x_format == "Month":
    plt.gca().xaxis.set_major_locator(mdates.MonthLocator(interval=x_tick_spacing))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
elif x_format == "Year":
    plt.gca().xaxis.set_major_locator(mdates.YearLocator(base=x_tick_spacing))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
elif x_format == "String":
    pass  # No special formatting for string x-axis
else:
    raise ValueError(
        f"Unrecognized x_format: {x_format}. Use 'Decimal', 'Percentage', 'Scientific', 'S
    )

# Date formats set their own locators above, and linear tick spacing would
# override the log locator
```

```
if x_format in ("Decimal", "Percentage", "Scientific"):
    plt.gca().xaxis.set_major_locator(MultipleLocator(x_tick_spacing))
plt.xlabel(x_label)

if x_tick_rotation != 0:
    # Line up the x-ticks with the labels when rotated
    plt.xticks(rotation=x_tick_rotation, ha="right")
else:
    plt.xticks(rotation=x_tick_rotation)

# Format Y axis
if y_tick_spacing == "Auto":
    max_value = -math.inf
    min_value = math.inf
    for col in y_plot_columns:
        if df[col].max() > max_value:
            max_value = df[col].max()

        if df[col].min() < min_value:
            min_value = df[col].min()

    raw_y_spacing = (max_value - min_value) / 10
    y_tick_spacing = round_to_nice_value(raw_y_spacing)

if y_format_decimal_places == "Auto":
    if isinstance(y_tick_spacing, float):
        y_tick_spacing_decimal_places = max(
            0, -int(math.floor(math.log10(abs(y_tick_spacing))))
        )
    else:
        y_tick_spacing_decimal_places = 0

    # PercentFormatter shows the value x 100, which needs 2 fewer decimals
    if y_format == "Percentage":
        y_tick_spacing_decimal_places = max(0, y_tick_spacing_decimal_places - 2)

    y_format_decimal_places = y_tick_spacing_decimal_places

if y_format == "Decimal":
    plt.gca().yaxis.set_major_formatter(
        FuncFormatter(lambda x, _: f"{x:,.{y_format_decimal_places}f}")
    )
elif y_format == "Percentage":
    plt.gca().yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=y_format_decimal_places)
    )
elif y_format == "Scientific":
    plt.gca().yaxis.set_major_formatter(
```

```
        FuncFormatter(lambda x, _: f"{x:.{y_format_decimal_places}e}")
    )
elif y_format == "Log":
    plt.yscale("log")
else:
    raise ValueError(
        f"Unrecognized y_format: {y_format}. Use 'Decimal', 'Percentage', 'Scientific', or
    )

# Linear tick spacing would override the log locator
if y_format != "Log":
    plt.gca().yaxis.set_major_locator(MultipleLocator(y_tick_spacing))

plt.ylabel(y_label)
plt.yticks(rotation=y_tick_rotation)

if plot_OLS_regression_line == True:

    model = run_regression(
        df=df,
        x_plot_column=x_plot_column,
        y_plot_column=OLS_column,
        regression_model="OLS-sklearn",
        regression_constant=regression_constant,
    )

    # Calc X and Y values for regression line
    X_vals = np.linspace(df[x_plot_column].min(), df[x_plot_column].max(), 100)
    X_vals_2d = X_vals.reshape(-1, 1)  # sklearn needs 2D input
    Y_vals = model.predict(X_vals_2d)

    # Plot regression line
    intercept = model.intercept_
    slope = model.coef_[0]
    # Score on the same rows the model was fit on (run_regression drops NaN)
    scored = df[[x_plot_column, OLS_column]].dropna()
    r_squared = model.score(scored[[x_plot_column]], scored[OLS_column])
    plt.plot(
        X_vals,
        Y_vals,
        color="red",
        linestyle="--",
        label=f"OLS Fit: y = {intercept:.1f} + {slope:.2f}x (R² = {r_squared:.2f})",
    )

if plot_Ridge_regression_line == True:

    model = run_regression(
```

```
            df=df,
            x_plot_column=x_plot_column,
            y_plot_column=Ridge_column,
            regression_model="Ridge",
            regression_constant=regression_constant,
            ridge_alpha=1.0,
        )

        # Calc X and Y values for regression line
        X_vals = np.linspace(df[x_plot_column].min(), df[x_plot_column].max(), 100)
        X_vals_2d = X_vals.reshape(-1, 1)  # sklearn needs 2D input
        Y_vals = model.predict(X_vals_2d)

        # Plot regression line
        intercept = model.intercept_
        slope = model.coef_[0]
        alpha_value = model.alpha_ if hasattr(model, "alpha_") else 1.0
        # Score on the same rows the model was fit on (run_regression drops NaN)
        scored = df[[x_plot_column, Ridge_column]].dropna()
        r_squared = model.score(scored[[x_plot_column]], scored[Ridge_column])
        plt.plot(
            X_vals,
            Y_vals,
            color="blue",
            linestyle="--",
            label=f"Ridge Fit ( ={alpha_value:.2f}): y = {intercept:.1f} + {slope:.2f}x (R² = {
        )

    if plot_RidgeCV_regression_line == True:

        model = run_regression(
            df=df,
            x_plot_column=x_plot_column,
            y_plot_column=RidgeCV_column,
            regression_model="RidgeCV",
            regression_constant=regression_constant,
        )

        # Calc X and Y values for regression line
        X_vals = np.linspace(df[x_plot_column].min(), df[x_plot_column].max(), 100)
        X_vals_2d = X_vals.reshape(-1, 1)  # sklearn needs 2D input
        Y_vals = model.predict(X_vals_2d)

        # Plot regression line
        intercept = model.intercept_
        slope = model.coef_[0]
        alpha_value = model.alpha_ if hasattr(model, "alpha_") else 1.0
        # Score on the same rows the model was fit on (run_regression drops NaN)
```

```
        scored = df[[x_plot_column, RidgeCV_column]].dropna()
        r_squared = model.score(scored[[x_plot_column]], scored[RidgeCV_column])
        plt.plot(
            X_vals,
            Y_vals,
            color="green",
            linestyle="--",
            label=f"RidgeCV Fit ( ={alpha_value:.2f}): y = {intercept:.1f} + {slope:.2f}x (R² =
        )

    # Format title, layout, grid, and legend
    plt.title(title)
    plt.tight_layout()

    # Grid
    if grid == True:
        plt.grid(True, linestyle="--", alpha=0.5)

    # Legend
    if legend == True:
        if legend_anchor is not None:
            plt.legend(loc=legend_location, bbox_to_anchor=legend_anchor)
        else:
            plt.legend(loc=legend_location)

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)

    # Save figure and display plot
    if export_plot == True:
        plt.savefig(f"{plot_file_name}.png", dpi=300, bbox_inches="tight")

    # Display the plot
    plt.show()

    return None
```

0.6.31 plot_scatter_regression_ffr_vs_returns

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm


def plot_scatter_regression_ffr_vs_returns(
```

```
    cycle_df: pd.DataFrame,
    asset_label: str,
    x_vals: np.ndarray,
    y_vals: np.ndarray,
    intercept: float,
    slope: float,
    caption: str = None,
) -> None:
    """
    Plot a scatter plot of annualized returns vs annualized change in Fed Funds
    Rate, with a regression line and annotations for each policy cycle.

    Parameters
    ----------
    cycle_df : pd.DataFrame
        DataFrame containing the policy cycle data with columns 'FFR_AnnualizedChange_bps', 'A
    asset_label : str
        Label for the asset being plotted (e.g., "S&P 500").
    x_vals : np.ndarray
        Array of x values for the regression line.
    y_vals : np.ndarray
        Array of y values for the regression line.
    intercept : float
        Intercept of the OLS regression line.
    slope : float
        Slope of the OLS regression line.
    caption : str, optional
        Caption drawn centered below the figure, e.g. "Figure 1" (default is
        None).

    Returns
    -------
    None
    """

    # Set plot figure size and background color
    plt.figure(figsize=(10, 6))

    # Plot data
    sns.scatterplot(
        data=cycle_df,
        x="FFR_AnnualizedChange_bps",
        y="AnnualizedReturnPct",
        s=100,
        color="blue",
    )

    # Annotate each point with the cycle number or date range, annualized returns and FFR
```

```
    for i, row in cycle_df.iterrows():
        plt.text(
            row["FFR_AnnualizedChange_bps"] + 5,  # small x-offset
            row["AnnualizedReturnPct"],
            row["Cycle"],
            # fontsize=10,
            color="black",
        )

    plt.plot(
        x_vals,
        y_vals,
        color="red",
        linestyle="--",
        label=f"OLS Fit: y = {intercept:.1f} + {slope:.2f}x",
    )
    plt.axhline(0, color="gray", linestyle="--", linewidth=0.8)
    plt.axvline(0, color="gray", linestyle="--", linewidth=0.8)

    # Format X axis
    plt.xlabel("Annualized Change In Fed Funds Rate (bps)")
    plt.xticks()

    # Format Y axis
    plt.ylabel(f"{asset_label} Annualized Return (%)")
    plt.yticks()

    # Format title, layout, grid, and legend
    plt.title(
        f"{asset_label} Annualized Return vs Annualized Change in Fed Funds Rate by Policy Cycl
    )
    plt.tight_layout()
    plt.grid(True)
    plt.legend()

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)

    # Display the plot
    plt.show()

    return None
```

0.6.32 plot_stats

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
from matplotlib.ticker import MultipleLocator


def plot_stats(
    stats_df: pd.DataFrame,
    plot_columns,
    title: str,
    x_label: str,
    x_tick_rotation: int,
    x_tick_spacing: int,
    y_label: str,
    y_tick_spacing: int,
    grid: bool,
    legend: bool,
    export_plot: bool,
    plot_file_name: str,
    caption: str = None,
) -> None:
    """
    Plot the price data from a DataFrame for a specified date range and columns.

    Parameters:
    -----------
    stats_df : pd.DataFrame
        DataFrame containing the price data to plot.
    plot_columns : str OR list
        List of columns to plot from the DataFrame. If none, all columns will be plotted.
    title : str
        Title of the plot.
    x_label : str
        Label for the x-axis.
    x_tick_rotation : int
        Rotation angle for the x-axis date labels.
    x_tick_spacing : int
        Spacing for the x-axis ticks.
    y_label : str
        Label for the y-axis.
    y_tick_spacing : int
        Spacing for the y-axis ticks.
    grid : bool
        Whether to display a grid on the plot.
    legend : bool
        Whether to display a legend on the plot.
    export_plot : bool
        Whether to save the figure as a PNG file.
    plot_file_name : str
        File name for saving the figure (if save_fig is True).
```

```
    caption : str, optional
        Caption drawn centered below the figure, e.g. "Figure 1" (default
        is None).

    Returns:
    --------
    None
    """

    # Set plot figure size and background color
    plt.figure(figsize=(10, 6))

    # Plot data
    if plot_columns == "All":
        for col in stats_df.columns:
            plt.scatter(stats_df.index, stats_df[col], label=col, marker="o", alpha=0.7)
    else:
        for col in plot_columns:
            plt.scatter(stats_df.index, stats_df[col], label=col, marker="o", alpha=0.7)

    # Format X axis
    plt.gca().xaxis.set_major_locator(MultipleLocator(x_tick_spacing))
    plt.xlabel(x_label)

    if x_tick_rotation != 0:
        # Line up the x-ticks with the labels when rotated
        plt.xticks(rotation=x_tick_rotation, ha="right")
    else:
        plt.xticks(rotation=x_tick_rotation)

    # Format Y axis
    plt.gca().yaxis.set_major_locator(MultipleLocator(y_tick_spacing))
    plt.ylabel(y_label)
    plt.yticks()

    # Format title, layout, grid, and legend
    plt.title(title)
    plt.tight_layout()

    if grid == True:
        plt.grid(True, linestyle="--", alpha=0.7)

    if legend == True:
        plt.legend()

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)
```

```
    # Save figure and display plot
    if export_plot == True:
        plt.savefig(f"{plot_file_name}.png", dpi=300, bbox_inches="tight")

    # Display the plot
    plt.show()

    return None
```

0.6.33 plot_time_series

```
import math
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd

from matplotlib.ticker import (
    FuncFormatter,
    MultipleLocator,
    PercentFormatter,
)

from round_to_nice_value import round_to_nice_value


def plot_time_series(
    df: pd.DataFrame,
    plot_start_date: str,
    plot_end_date: str,
    plot_columns: str | list[str],
    title: str,
    x_label: str,
    x_format: str,
    x_tick_spacing: int,
    x_tick_start: str,
    x_tick_rotation: int,
    y_label: str,
    y_format: str,
    y_format_decimal_places: int | str,
    y_tick_spacing: int | float | str,
    y_tick_rotation: int,
    emphasis_columns: str | list[str] = None,
    emphasis_linewidth: float = 3.0,
    emphasis_alpha: float = 1.0,
    grid: bool = True,
    legend: bool = True,
    legend_location: str = "best",
```

```
    legend_anchor: tuple = None,
    export_plot: bool = False,
    plot_file_name: str = None,
    caption: str = None,
) -> None:
    """
    Plot the time series data from a DataFrame for a specified date range and columns.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame containing the time series data to plot.
    plot_start_date : str
        Start date for the plot in 'YYYY-MM-DD' format.
    plot_end_date : str
        End date for the plot in 'YYYY-MM-DD' format.
    plot_columns : str OR list
        Column or list of columns to plot from the DataFrame, or "All" to plot
        every column.
    title : str
        Title of the plot.
    x_label : str
        Label for the x-axis.
    x_format : str
        Format for the x-axis date labels.
    x_tick_spacing : int
        Spacing for the x-axis ticks.
    x_tick_start : str
        Start date for the x-axis ticks.
    x_tick_rotation : int
        Rotation angle for the x-axis tick labels.
    y_label : str
        Label for the y-axis.
    y_format : str
        Format for the y-axis labels: "Decimal", "Percentage", "Scientific", or
        "Log".
    y_format_decimal_places : int OR str
        Number of decimal places for y-axis labels, or "Auto" to infer it from
        the y-axis tick spacing.
    y_tick_spacing : int OR float OR str
        Spacing for the y-axis ticks, or "Auto" to derive it from the range of
        the plotted data. Ignored when y_format is "Log".
    y_tick_rotation : int
        Rotation angle for the y-axis tick labels.
    emphasis_columns : str OR list, optional
        Column(s) to draw with emphasis: full opacity and drawn on top of the
        other series, plus a heavier line weight if emphasis_linewidth is
        raised. Colors are unchanged, and the non-emphasized series keep their
```

```
    normal styling (default is None).
emphasis_linewidth : float, optional
    Line width used for emphasized columns (default is 3.0).
emphasis_alpha : float, optional
    Opacity used for emphasized columns (default is 1.0).
grid : bool, optional
    Whether to display a grid on the plot (default is True).
legend : bool, optional
    Whether to display a legend on the plot (default is True).
legend_location : str, optional
    Location of the legend on the plot (default is "best").
legend_anchor : tuple, optional
    Anchor point (x, y) for placing the legend relative to the axes,
    e.g. (1, 1) to move it outside the plot (default is None).
export_plot : bool, optional
    Whether to save the figure as a PNG file (default is False).
plot_file_name : str, optional
    File name for saving the figure (if export_plot is True, default is None).
caption : str, optional
    Caption drawn centered below the figure, e.g. "Figure 1" (default is
    None).

Returns
-------
None
"""

# If start date and end date are None, use the entire DataFrame
if plot_start_date is None and plot_end_date is None:
    df_filtered = df.copy()

# If only end date is specified, filter by end date
elif plot_start_date is None and plot_end_date is not None:
    df_filtered = df[(df.index <= plot_end_date)].copy()

# If only start date is specified, filter by start date
elif plot_start_date is not None and plot_end_date is None:
    df_filtered = df[(df.index >= plot_start_date)].copy()

# If both start date and end date are specified, filter by both
else:
    df_filtered = df[
        (df.index >= plot_start_date) & (df.index <= plot_end_date)
    ].copy()

# Set plot figure size and background color
plt.figure(figsize=(10, 6))
```

```
# Plot data
cols = df_filtered.columns if plot_columns == "All" else plot_columns

# Normalize emphasis columns to a set for membership testing
if emphasis_columns is None:
    emphasis_cols = set()
elif isinstance(emphasis_columns, str):
    emphasis_cols = {emphasis_columns}
else:
    emphasis_cols = set(emphasis_columns)

# Catch typos rather than silently plotting without emphasis
unrecognized = emphasis_cols - set(cols)
if unrecognized:
    raise ValueError(
        f"Unrecognized emphasis_columns: {sorted(unrecognized)}. "
        f"Columns being plotted: {sorted(cols)}."
    )

for col in cols:
    emphasized = col in emphasis_cols
    plt.plot(
        df_filtered.index,
        df_filtered[col],
        label=col,
        linestyle="-",
        linewidth=emphasis_linewidth if emphasized else 1.5,
        alpha=emphasis_alpha if emphasized else 0.5,
        # Layer emphasized series on top without reordering the loop, which
        # would shift the color cycle and the legend order
        zorder=3 if emphasized else 2,
    )

# Format X axis
if x_format == "Second":
    if x_tick_start is not None:
        ticks = pd.date_range(
            start=pd.Timestamp(x_tick_start).tz_convert(df_filtered.index.tz),
            end=df_filtered.index.max() + pd.tseries.offsets.Second(x_tick_spacing),
            freq=pd.tseries.offsets.Second(x_tick_spacing),
        )
        plt.gca().set_xticks(ticks)
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M:%S"))
    else:
        plt.gca().xaxis.set_major_locator(
            mdates.SecondLocator(interval=x_tick_spacing)
        )
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M:%S"))
```

```
elif x_format == "Minute":
    if x_tick_start is not None:
        ticks = pd.date_range(
            start=pd.Timestamp(x_tick_start).tz_convert(df_filtered.index.tz),
            end=df_filtered.index.max() + pd.tseries.offsets.Minute(x_tick_spacing),
            freq=pd.tseries.offsets.Minute(x_tick_spacing),
        )
        plt.gca().set_xticks(ticks)
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    else:
        plt.gca().xaxis.set_major_locator(
            mdates.MinuteLocator(interval=x_tick_spacing)
        )
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
elif x_format == "Hour":
    if x_tick_start is not None:
        ticks = pd.date_range(
            start=pd.Timestamp(x_tick_start).tz_convert(df_filtered.index.tz),
            end=df_filtered.index.max() + pd.tseries.offsets.Hour(x_tick_spacing),
            freq=pd.tseries.offsets.Hour(x_tick_spacing),
        )
        plt.gca().set_xticks(ticks)
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    else:
        plt.gca().xaxis.set_major_locator(
            mdates.HourLocator(interval=x_tick_spacing)
        )
        plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
elif x_format == "Day":
    plt.gca().xaxis.set_major_locator(mdates.DayLocator(interval=x_tick_spacing))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%d %b %Y"))
elif x_format == "Week":
    plt.gca().xaxis.set_major_locator(
        mdates.WeekdayLocator(interval=x_tick_spacing)
    )
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%d %b %Y"))
elif x_format == "Month":
    plt.gca().xaxis.set_major_locator(mdates.MonthLocator(interval=x_tick_spacing))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
elif x_format == "Year":
    plt.gca().xaxis.set_major_locator(mdates.YearLocator(base=x_tick_spacing))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
else:
    raise ValueError(
        f"Unrecognized x_format: {x_format}. Use 'Second', 'Minute', 'Hour', 'Day', 'Week'
    )

plt.xlabel(x_label)
```

```
if x_tick_rotation != 0:
    # Line up the x-ticks with the labels when rotated
    plt.xticks(rotation=x_tick_rotation, ha="right")
else:
    plt.xticks(rotation=x_tick_rotation)

# Format Y axis
if y_tick_spacing == "Auto":
    max_value = -math.inf
    min_value = math.inf
    cols = df_filtered.columns if plot_columns == "All" else plot_columns
    for col in cols:
        if df_filtered[col].max() > max_value:
            max_value = df_filtered[col].max()

        if df_filtered[col].min() < min_value:
            min_value = df_filtered[col].min()

    raw_y_spacing = (max_value - min_value) / 10
    y_tick_spacing = round_to_nice_value(raw_y_spacing)

if y_format_decimal_places == "Auto":
    if isinstance(y_tick_spacing, float):
        y_tick_spacing_decimal_places = max(
            0, -int(math.floor(math.log10(abs(y_tick_spacing))))
        )
    else:
        y_tick_spacing_decimal_places = 0

    # PercentFormatter shows the value x 100, which needs 2 fewer decimals
    if y_format == "Percentage":
        y_tick_spacing_decimal_places = max(0, y_tick_spacing_decimal_places - 2)

    y_format_decimal_places = y_tick_spacing_decimal_places

if y_format == "Decimal":
    plt.gca().yaxis.set_major_formatter(
        FuncFormatter(lambda x, _: f"{x:,.{y_format_decimal_places}f}")
    )
elif y_format == "Percentage":
    plt.gca().yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=y_format_decimal_places)
    )
elif y_format == "Scientific":
    plt.gca().yaxis.set_major_formatter(
        FuncFormatter(lambda x, _: f"{x:.{y_format_decimal_places}e}")
    )
```

```
    elif y_format == "Log":
        plt.yscale("log")
    else:
        raise ValueError(
            f"Unrecognized y_format: {y_format}. Use 'Decimal', 'Percentage', 'Scientific', or
        )

    # Linear tick spacing would override the log locator
    if y_format != "Log":
        plt.gca().yaxis.set_major_locator(MultipleLocator(y_tick_spacing))

    plt.ylabel(y_label)
    plt.yticks(rotation=y_tick_rotation)

    # Format title, layout, grid, and legend
    plt.title(title)
    plt.tight_layout()

    # Grid
    if grid == True:
        plt.grid(True, linestyle="--", alpha=0.5)

    # Legend
    if legend == True:
        if legend_anchor is not None:
            plt.legend(loc=legend_location, bbox_to_anchor=legend_anchor)
        else:
            plt.legend(loc=legend_location)

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)

    # Save figure and display plot
    if export_plot == True:
        plt.savefig(f"{plot_file_name}.png", dpi=300, bbox_inches="tight")

    # Display the plot
    plt.show()

    return None
```

0.6.34 plot_vix_with_trades

```
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd
```

```
from matplotlib.ticker import MultipleLocator


def plot_vix_with_trades(
    vix_price_df: pd.DataFrame,
    trades_df: pd.DataFrame,
    plot_start_date: str,
    plot_end_date: str,
    x_tick_spacing: int,
    y_tick_spacing: int,
    index_number: str,
    export_plot: bool,
    caption: str = None,
) -> pd.DataFrame:
    """
    Plot the VIX daily high and low prices, along with the VIX spikes, and trades.

    Parameters:
    -----------
    vix_price_df : pd.DataFrame
        Dataframe containing the VIX price data to plot.
    trades_df : pd.DataFrame
        Dataframe containing the trades data.
    plot_start_date : str
        Start date for the plot in 'YYYY-MM-DD' format.
    plot_end_date : str
        End date for the plot in 'YYYY-MM-DD' format.
    index_number : str
        Index number to be used in the file name of the plot export.
    export_plot : bool
        Whether to save the figure as a PNG file.
    caption : str, optional
        Caption drawn centered below the figure, e.g. "Figure 1" (default
        is None).

    Returns:
    --------
    vix_data : pd.DataFrame
        Dataframe containing the VIX price data for the specified timeframe.
    """

    # Create temporary dataframe for the specified date range
    vix_data = vix_price_df[
        (vix_price_df.index >= plot_start_date) & (vix_price_df.index <= plot_end_date)
    ]

    # Set plot figure size and background color
    plt.figure(figsize=(12, 6), facecolor="#F5F5F5")
```

```
# Plot VIX high and low price data
plt.plot(
    vix_data.index,
    vix_data["High"],
    label="High",
    linestyle="-",
    color="steelblue",
    linewidth=1,
)
plt.plot(
    vix_data.index,
    vix_data["Low"],
    label="Low",
    linestyle="-",
    color="brown",
    linewidth=1,
)

# Plot VIX spikes
plt.scatter(
    vix_data[vix_data["Spike_SMA"] == True].index,
    vix_data[vix_data["Spike_SMA"] == True]["High"],
    label="Spike (High > 1.25 * 10 Day High SMA)",
    color="black",
    s=20,
)

# Plot trades
plt.scatter(
    trades_df["Trade_Date"],
    trades_df["Approx_VIX_Level"],
    label="Trades",
    color="red",
    s=20,
)

# Annotate each point in trades_df with the corresponding Action_Symbol
for _, row in trades_df.iterrows():
    plt.text(
        row["Trade_Date"] + pd.Timedelta(days=1),
        row["Approx_VIX_Level"] + 0.1,
        row["TradeDate_Action_Symbol_VIX"],
        fontsize=9,
    )

# Format X axis
plt.gca().xaxis.set_major_locator(mdates.DayLocator(interval=x_tick_spacing))
```

```
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
    plt.xlabel("Date", fontsize=10)
    plt.xticks(rotation=45, fontsize=8)

    # Format Y axis
    plt.gca().yaxis.set_major_locator(MultipleLocator(y_tick_spacing))
    plt.ylabel("VIX", fontsize=10)
    plt.yticks(fontsize=8)

    # Format title, layout, grid, and legend
    plt.title(
        f"CBOE Volatility Index (VIX), VIX Spikes, Trades, {plot_start_date} - {plot_end_date}"
        fontsize=12,
    )
    plt.tight_layout()
    plt.grid(True, linestyle="--", alpha=0.7)
    plt.legend(fontsize=9)

    # Caption
    if caption is not None:
        plt.gcf().text(0.5, 0, caption, ha="center", va="top", fontsize=9)

    # Save figure and display plot
    if export_plot == True:
        # plt.savefig(f"{index_number}_VIX_Spike_Trades_{plot_start_date}_{plot_end_date}.png"
        plt.savefig(
            f"{index_number}_VIX_Spike_Trades.png", dpi=300, bbox_inches="tight"
        )

    # Display the plot
    plt.show()

    return vix_data
```

0.6.35 polygon_fetch_full_history

```
import pandas as pd
import time

from datetime import datetime, timedelta
from massive import RESTClient
from settings import config

# Load API key from the environment variables
POLYGON_KEY = config("POLYGON_KEY")

# Get the environment variable for where data is stored
DATA_DIR = config("DATA_DIR")
```

```python
def polygon_fetch_full_history(
    client,
    ticker: str,
    timespan: str,
    multiplier: int,
    adjusted: bool,
    existing_history_df: pd.DataFrame,
    current_start: datetime,
    free_tier: bool,
    verbose: bool,
) -> pd.DataFrame:
    """
    Fetch full historical data for a given product from Polygon API.

    Parameters:
    -----------
    client
        Polygon API client instance.
    ticker : str
        Ticker symbol to download.
    timespan : str
        Time span for the data (e.g., "minute", "hour", "day", "week", "month", "quarter", "ye
    multiplier : int
        Multiplier for the time span (e.g., 1 for daily data).
    adjusted : bool
        If True, return adjusted data; if False, return raw data.
    existing_history_df : pd.DataFrame
        DataFrame containing the existing historical data.
    current_start : datetime
        Date for which to start pulling data in datetime format.
    free_tier : bool
        If True, then pause to avoid API limits.
    verbose : bool
        If True, print detailed information about the data being processed.

    Returns:
    --------
    full_history_df : pd.DataFrame
        DataFrame containing the data.
    """

    # Copy DataFrame
    full_history_df = existing_history_df.copy()

    if timespan == "minute":
        time_delta = 15
```

```
    time_overlap = 1
elif timespan == "hour":
    time_delta = 15
    time_overlap = 1
elif timespan == "day":
    time_delta = 180
    time_overlap = 1
else:
    raise Exception(f"Invalid {timespan}.")

new_data_last_date = None
new_date_last_date_check = None

while current_start < datetime.now():

    # Offset end date by time_delta
    current_end = current_start + timedelta(days=time_delta)

    if verbose == True:
        print(
            f"Pulling {timespan} data for {current_start} thru {current_end} for {ticker}.
        )

    try:
        # Pull new data
        aggs = client.get_aggs(
            ticker=ticker,
            timespan=timespan,
            multiplier=multiplier,
            from_=current_start,
            to=current_end,
            adjusted=adjusted,
            sort="asc",
            limit=5000,
        )

        # if len(aggs) == 0:
        # raise Exception(f"No data is available for {ticker} for {current_start} thru {cu

        # Convert to DataFrame
        new_data = pd.DataFrame([bar.__dict__ for bar in aggs])
        new_data["timestamp"] = pd.to_datetime(new_data["timestamp"], unit="ms")
        new_data = new_data.rename(columns={"timestamp": "Date"})
        new_data = new_data[
            [
                "Date",
                "open",
                "high",
```

```
        "low",
        "close",
        "volume",
        "vwap",
        "transactions",
        "otc",
    ]
]
new_data = new_data.sort_values(by="Date", ascending=True)

# Enforce dtypes to match full_history_df
new_data = new_data.astype(full_history_df.dtypes.to_dict())

# (Optional) reorder columns to match schema
# new_data = new_data[full_history_df.columns]

# Find last date in new_data
new_data_last_date = new_data["Date"].max()

if verbose == True:
    print("New data:")
    print(new_data)

# No longer necessary to check for 5000 rows of data

# Check if new data contains 5000 rows
# if len(new_data) == 5000:
# raise Exception(f"New data for {ticker} contains 5000 rows, indicating potential

# If full_history_df length is not 0, check to confirm that data overlaps to verif
# if not full_history_df.empty:
# if not full_history_df['Date'].isin(new_data['Date']).any():
# raise Exception(f"New data does not overlap with existing data.")

if not full_history_df.empty:
    # Columns present in both frames
    common_cols = list(
        full_history_df.columns.intersection(new_data.columns)
    )
    if not common_cols:
        raise Exception("No common columns to compare.")

    # (Optional) de-duplicate to speed up the merge
    # full_dedup = full_history_df[common_cols].drop_duplicates()
    # new_dedup  = new_data[common_cols].drop_duplicates()

    price_cols = ["open", "high", "low", "close"]
```

```
            # Inner join on every shared column = exact row matches
            # overlap = full_dedup.merge(new_dedup, on=common_cols, how="inner")
            # overlap = full_dedup.merge(new_dedup, on=price_cols, how="inner")
            overlap = full_history_df.merge(new_data, on=price_cols, how="inner")

            if overlap.empty:
                raise Exception(
                    # f"New data does not overlap with existing data (full-row check)."
                    f"New data does not overlap with existing data (price data check)."
                )

        # Combine existing data with recent data, drop duplicates, sort values, reset index
        full_history_df = pd.concat([full_history_df, new_data])
        full_history_df = full_history_df.drop_duplicates(
            subset="Date", keep="last"
        )
        full_history_df = full_history_df.sort_values(by="Date", ascending=True)
        full_history_df = full_history_df.reset_index(drop=True)

        if verbose == True:
            print("Combined data:")
            print(full_history_df)

    except KeyError as e:
        print(
            f"No data is available for {ticker} from {current_start} thru {current_end}."
        )

        user_choice = input(f"Press Enter to continue, or type 'q' to quit: ")
        if user_choice.lower() == "q":
            print(f"Aborting operation to update {ticker} {timespan} data.")
            break  # break out of the while loop
        else:
            print(f"Trying next timeframe for {ticker} {timespan} data.")

            # Set last_data_date to current_end because we know data was not available
            # up until current_end
            new_data_last_date = current_end
            pass

    except Exception as e:
        print(
            f"Failed to pull {timespan} data for {current_start} thru {current_end} for {ti
        )
        raise  # Re-raise the original exception

    # Break out of loop if data is up-to-date (or close to being up-to-date because it is
    # possible that entire range was not pulled due to the way API handles hour data
```

```
            # from minute data)
            if current_end > datetime.now():
                break
            else:
                # Edge case, if the last date for new_date is exactly time_overlap's duration
                # past current_start
                if new_date_last_date_check == new_data_last_date:
                    current_start = current_end - timedelta(days=time_overlap)
                else:
                    current_start = new_data_last_date - timedelta(days=time_overlap)

                new_date_last_date_check = new_data_last_date

                # Code below is likely not necessary

                # # Overlap with existing data to capture all data but check to see if
                # # current_end is a Sunday and if so ensure overlap covers a trading day
                # if current_end.weekday() == 6:
                #     current_start = last_data_date - timedelta(days=(time_overlap+1))
                # else:
                #     current_start = last_data_date - timedelta(days=time_overlap)

            # Check for free tier and if so then pause for 12 seconds to avoid hitting API rate li
            if free_tier == True:
                if verbose == True:
                    print(f"Sleeping for 12 seconds to avoid hitting API rate limits...\n")
                time.sleep(12)

    # Return the DataFrame containing the full history
    return full_history_df


if __name__ == "__main__":

    current_year = datetime.now().year
    current_month = datetime.now().month
    current_day = datetime.now().day

    # Open client connection
    client = RESTClient(api_key=POLYGON_KEY)

    # Create an empty DataFrame
    df = pd.DataFrame(
        {
            "Date": pd.Series(dtype="datetime64[ns]"),
            "open": pd.Series(dtype="float64"),
            "high": pd.Series(dtype="float64"),
            "low": pd.Series(dtype="float64"),
```

```
            "close": pd.Series(dtype="float64"),
            "volume": pd.Series(dtype="float64"),
            "vwap": pd.Series(dtype="float64"),
            "transactions": pd.Series(dtype="int64"),
            "otc": pd.Series(dtype="object"),
        }
    )

    # Example usage - minute
    df = polygon_fetch_full_history(
        client=client,
        ticker="TQQQ",
        timespan="hour",
        multiplier=1,
        adjusted=True,
        existing_history_df=df,
        current_start=datetime(current_year - 2, current_month, current_day),
        free_tier=True,
        verbose=True,
    )
```

0.6.36 polygon_fetch_ticker_details

```
import pandas as pd
import time

from massive import RESTClient
from settings import config

# Load API key from the environment variables
POLYGON_KEY = config("POLYGON_KEY")

# Get the environment variable for where data is stored
DATA_DIR = config("DATA_DIR")

# Open client connection
client = RESTClient(api_key=POLYGON_KEY)


def polygon_fetch_ticker_details(
    ticker: str,
) -> any:
    """
    Fetch detailed information for a given product from Polygon API.

    Parameters:
    -----------
    ticker : str
```

```
        Ticker symbol to download.

    Returns:
    --------
    details : dict
        Dictionary containing detailed information about the ticker.
    """

    details = client.get_ticker_details(ticker)

    return details


if __name__ == "__main__":

    ################
    # Stock data
    ################

    new_equity_tickers = [
        # Put new tickers here that you want to pull details for
    ]

    # Create empty dictionary
    new_equity = {}

    # Read existing data from csv file into equities dictionary
    equities_df = pd.read_csv(f"{DATA_DIR}/Polygon/equities.csv", index_col=0)
    equities = equities_df.to_dict()["Name"]

    for ticker in new_equity_tickers:
        ticker_details = polygon_fetch_ticker_details(ticker)
        new_equity[ticker] = ticker_details.name  # access name field
        equities.update(new_equity)
        equities = dict(sorted(equities.items()))
        print(f"Updated {ticker}")
        time.sleep(12)

    print(equities)

    # Export equities dictionary to csv file
    equities_df = pd.DataFrame.from_dict(equities, orient="index", columns=["Name"])
    equities_df.to_csv(f"{DATA_DIR}/Polygon/equities.csv")

    time.sleep(12)

    ################
    # ETF data
```

```
    ###############

    new_etf_tickers = [
        # Put new tickers here that you want to pull details for
    ]

    # Create empty dictionary
    new_etf = {}

    # Read existing data from csv file into etfs dictionary
    etfs_df = pd.read_csv(f"{DATA_DIR}/Polygon/etfs.csv", index_col=0)
    etfs = etfs_df.to_dict()["Name"]

    for ticker in new_etf_tickers:
        ticker_details = polygon_fetch_ticker_details(ticker)
        new_etf[ticker] = ticker_details.name  # access name field
        etfs.update(new_etf)
        etfs = dict(sorted(etfs.items()))
        print(f"Updated {ticker}")
        time.sleep(12)

    print(etfs)

    # Export etfs dictionary to csv file
    etfs_df = pd.DataFrame.from_dict(etfs, orient="index", columns=["Name"])
    etfs_df.to_csv(f"{DATA_DIR}/Polygon/etfs.csv")
```

0.6.37 polygon_pull_data

```
import os
import pandas as pd
import time

from datetime import datetime, timedelta
from IPython.display import display
from massive import RESTClient
from polygon_fetch_full_history import polygon_fetch_full_history
from settings import config

# Load API key from the environment variables
POLYGON_KEY = config("POLYGON_KEY")

# Get the environment variable for where data is stored
DATA_DIR = config("DATA_DIR")


def polygon_pull_data(
    base_directory,
```

```
    ticker: str,
    source: str,
    asset_class: str,
    start_date: datetime,
    timespan: str,
    multiplier: int,
    adjusted: bool,
    force_existing_check: bool,
    free_tier: bool,
    verbose: bool,
    excel_export: bool,
    pickle_export: bool,
    parquet_export: bool,
    output_confirmation: bool,
) -> pd.DataFrame:
    """
    Read existing data file, download price data from Polygon, and export data.

    Parameters:
    -----------
    base_directory : any
        Root path to store downloaded data.
    ticker : str
        Ticker symbol to download.
    source : str
        Name of the data source (e.g., 'Polygon').
    asset_class : str
        Asset class name (e.g., 'Equities').
    start_date : datetime
        Start date for the data in datetime format.
    timespan : str
        Time span for the data (e.g., "minute", "hour", "day", "week", "month", "quarter", "ye
    multiplier : int
        Multiplier for the time span (e.g., 1 for daily data).
    adjusted : bool
        If True, return adjusted data; if False, return raw data.
    force_existing_check : bool
        If True, force a complete check of the existing data file to verify that there are not
    free_tier : bool
        If True, then pause to avoid API limits.
    verbose : bool
        If True, print detailed information about the data being processed.
    excel_export : bool
        If True, export data to Excel format.
    pickle_export : bool
        If True, export data to Pickle format.
    parquet_export : bool
        If True, export data to Parquet format.
```

```
    output_confirmation : bool
        If True, print confirmation message.

    Returns:
    --------
    pd.DataFrame
        DataFrame containing the updated price data.
    """

    # Open client connection
    client = RESTClient(api_key=POLYGON_KEY)

    # Set file location based on parameters
    file_location = f"{base_directory}/{source}/{asset_class}/{timespan}/{ticker}.pkl"

    # Create list of acceptable timespans
    acceptable_timespans = ["minute", "hour", "day"]
    if timespan not in acceptable_timespans:
        raise Exception(
            f"Invalid timespan: {timespan}. Acceptable timespans are: {acceptable_timespans}."
        )

    try:
        # Attempt to read existing pickle data file
        existing_history_df = pd.read_pickle(file_location)

        # Reset index if 'Date' is column is the index
        if "Date" not in existing_history_df.columns:
            existing_history_df = existing_history_df.reset_index()

        print(f"File found...updating the {ticker} {timespan} data.")

        if verbose == True:
            print("Existing data:")
            print(existing_history_df)

        # Find last date in existing data
        last_data_date = existing_history_df["Date"].max()
        print(f"Last date in existing data: {last_data_date}")

        # Find the number of starting rows
        starting_rows = len(existing_history_df)
        print(f"Number of rows in existing data: {starting_rows}")

        # Overlap with existing data to capture all data
        # current_start = last_data_date - timedelta(days=time_overlap)
        current_start = last_data_date - timedelta(days=1)
```

```
except FileNotFoundError:
    # Print error
    print(f"File not found...downloading the {ticker} {timespan} data.")

    # Create an empty DataFrame
    existing_history_df = pd.DataFrame(
        {
            "Date": pd.Series(dtype="datetime64[ns]"),
            "open": pd.Series(dtype="float64"),
            "high": pd.Series(dtype="float64"),
            "low": pd.Series(dtype="float64"),
            "close": pd.Series(dtype="float64"),
            "volume": pd.Series(dtype="float64"),
            "vwap": pd.Series(dtype="float64"),
            "transactions": pd.Series(dtype="int64"),
            "otc": pd.Series(dtype="object"),
        }
    )

    # Set the number of starting rows to be 0
    starting_rows = 0

    # Set current date to start date
    current_start = start_date

# Check for force_existing_check flag
if force_existing_check == True:
    print("Forcing check of existing data...")
    current_start = start_date

full_history_df = polygon_fetch_full_history(
    client=client,
    ticker=ticker,
    timespan=timespan,
    multiplier=multiplier,
    adjusted=adjusted,
    existing_history_df=existing_history_df,
    current_start=current_start,
    free_tier=free_tier,
    verbose=verbose,
)

# Create directory
directory = f"{base_directory}/{source}/{asset_class}/{timespan}"
os.makedirs(directory, exist_ok=True)

# Export to Excel
if excel_export == True:
```

```
            print(f"Exporting {ticker} {timespan} data to Excel...")
            full_history_df.to_excel(f"{directory}/{ticker}.xlsx", sheet_name="data")

        # Export to Pickle
        if pickle_export == True:
            print(f"Exporting {ticker} {timespan} data to Pickle...")
            full_history_df.to_pickle(f"{directory}/{ticker}.pkl")

        # Export to Parquet
        if parquet_export == True:
            print(f"Exporting {ticker} {timespan} data to Parquet...")
            full_history_df.to_parquet(f"{directory}/{ticker}.parquet")

        total_rows = len(full_history_df)

        # Output confirmation
        if output_confirmation == True:
            print(f"The first and last date of {timespan} data for {ticker} is: ")
            display(full_history_df[:1])
            display(full_history_df[-1:])
            print(f"Number of rows after data update: {total_rows}")

            if starting_rows:
                print(f"Number of rows added during update: {total_rows - starting_rows}")

            print(f"Polygon data complete for {ticker} {timespan} data.")
            print(f"--------------------")

        return full_history_df


if __name__ == "__main__":

    # Get current year, month, day
    current_year = datetime.now().year
    current_month = datetime.now().month
    current_day = datetime.now().day

    # Set global variables
    GLOBAL_FREE_TIER = False

    # Stock Data
    equities = ["BKNG"]

    # Timespans
    timespans = ["day"]

    # Iterate through each stock
```

```
    for stock in equities:
        for timespan in timespans:
            polygon_pull_data(
                base_directory=DATA_DIR,
                ticker=stock,
                source="Polygon",
                asset_class="Equities",
                start_date=datetime(current_year - 5, current_month, current_day),
                timespan=timespan,
                multiplier=1,
                adjusted=False,
                force_existing_check=False,
                free_tier=GLOBAL_FREE_TIER,
                verbose=False,
                excel_export=True,
                pickle_export=True,
                parquet_export=True,
                output_confirmation=True,
            )

            if GLOBAL_FREE_TIER == True:
                time.sleep(12)
            else:
                pass
```

0.6.38 run_regression

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

from sklearn.linear_model import (
    LinearRegression,
    LogisticRegression,
    LogisticRegressionCV,
    Ridge,
    RidgeCV,
)


def run_regression(
    df: pd.DataFrame,
    x_plot_column: str,
    y_plot_column: str,
    regression_model: str,
    regression_constant: bool,
    cov_type: str = None,
    cov_kwds: dict = None,
```

```
    ridge_alpha: float = None,
    logistic_C: float = 1.0,
    logistic_max_iter: int = 1000,
    logistic_solver: str = "lbfgs",
    logistic_cv_Cs: int = 10,
) -> (
    LinearRegression
    | Ridge
    | RidgeCV
    | LogisticRegression
    | LogisticRegressionCV
    | sm.regression.linear_model.RegressionResultsWrapper
):
    """
    Run a linear regression on the specified DataFrame columns, model,
    and constant.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame containing the data to fit.
    x_plot_column : str
        Column to use as the feature (X) from the DataFrame.
    y_plot_column : str
        Column to use as the target (y) from the DataFrame.
    regression_model : str
        Type of regression model to use. One of:
            "OLS-statsmodels", "OLS-sklearn", "Ridge", "RidgeCV",
            "Logistic", "LogisticCV".
    regression_constant : bool
        Whether to include a constant/intercept term in the model.
    cov_type : str, optional
        Covariance estimator passed to statsmodels' ``.fit()`` for the
        "OLS-statsmodels" model (e.g. "HAC" for Newey-West heteroskedasticity-
        and autocorrelation-consistent standard errors). If None (default), the
        standard non-robust covariance is used. Passing it with any other
        model raises a ValueError.
    cov_kwds : dict, optional
        Keyword arguments for the chosen ``cov_type`` (e.g. {"maxlags": 12} for
        "HAC"). Only used when ``cov_type`` is set.
    ridge_alpha : float, optional
        Alpha value for Ridge regression. Required if regression_model is "Ridge".
    logistic_C : float, optional
        Inverse of regularization strength for LogisticRegression. Smaller
        values produce stronger regularization. Default is 1.0.
    logistic_max_iter : int, optional
        Maximum number of iterations for the solver. Default is 1000.
    logistic_solver : str, optional
```

```
    Solver algorithm for LogisticRegression and LogisticRegressionCV.
    Default is "lbfgs". Other options: "liblinear", "saga", "newton-cg".
logistic_cv_Cs : int, optional
    Number of C values to try in LogisticRegressionCV, spaced
    logarithmically. Default is 10.

Returns
-------
model : LinearRegression | Ridge | RidgeCV | LogisticRegression
        | LogisticRegressionCV | RegressionResultsWrapper
    The fitted regression model.

Notes
-----
Logistic models expect a binary (0/1) or multiclass integer target column.
They are classification models, not regression models, so the interpretation
of coefficients differs from the linear models above.
"""

if cov_type is not None and regression_model != "OLS-statsmodels":
    raise ValueError(
        f"cov_type is only supported for 'OLS-statsmodels', "
        f"not '{regression_model}'."
    )

# Align X and y (common to all models)
data = df[[x_plot_column, y_plot_column]].dropna()
X = data[[x_plot_column]]
y = data[y_plot_column]

if regression_model == "OLS-statsmodels":
    if regression_constant:
        X_sm = sm.add_constant(X)
    else:
        X_sm = X
    if cov_type is not None:
        model = sm.OLS(y, X_sm).fit(cov_type=cov_type, cov_kwds=cov_kwds)
    else:
        model = sm.OLS(y, X_sm).fit()

elif regression_model == "OLS-sklearn":
    model = LinearRegression(fit_intercept=regression_constant)
    model.fit(X, y)

elif regression_model == "Ridge":
    model = Ridge(alpha=ridge_alpha, fit_intercept=regression_constant)
    model.fit(X, y)
```

```
    elif regression_model == "RidgeCV":
        alphas = np.logspace(-2, 1, 50)  # 50 values, 0.01 to 10
        model = RidgeCV(alphas=alphas, cv=5, fit_intercept=regression_constant)
        model.fit(X, y)

    elif regression_model == "Logistic":
        model = LogisticRegression(
            C=logistic_C,
            fit_intercept=regression_constant,
            max_iter=logistic_max_iter,
            solver=logistic_solver,
        )
        model.fit(X, y)

    elif regression_model == "LogisticCV":
        # Cs can be an int (number of log-spaced values) or an explicit array;
        # passing the int is the idiomatic sklearn approach here.
        model = LogisticRegressionCV(
            Cs=logistic_cv_Cs,
            fit_intercept=regression_constant,
            max_iter=logistic_max_iter,
            solver=logistic_solver,
            cv=5,
        )
        model.fit(X, y)

    else:
        raise ValueError(
            f"Unrecognized regression_model: '{regression_model}'. "
            "Supported models: 'OLS-statsmodels', 'OLS-sklearn', 'Ridge', "
            "'RidgeCV', 'Logistic', 'LogisticCV'."
        )

    return model
```

0.6.39 strategy_harry_brown_perm_port

```
import pandas as pd


def strategy_harry_brown_perm_port(
    fund_list: str,
    starting_cash: int,
    cash_contrib: int,
    close_prices_df: pd.DataFrame,
    rebal_month: int,
    rebal_day: int,
    rebal_per_high: float,
```

```
    rebal_per_low: float,
    excel_export: bool,
    pickle_export: bool,
    output_confirmation: bool,
    target_weights: dict = None,
) -> pd.DataFrame:
    """
    Execute the re-balance strategy based on specified criteria.

    Parameters:
    -----------
    fund_list (str):
        List of funds for data to be combined from. Funds are strings in the form "BTC-USD".
    starting_cash (int):
        Starting investment balance.
    cash_contrib (int):
        Cash contribution to be made daily.
    close_prices_df (pd.DataFrame):
        DataFrame containing date and close prices for all funds to be included.
    rebal_month (int):
        Month for annual rebalance.
    rebal_day (int):
        Day for annual rebalance.
    rebal_per_high (float):
        High rebalance band, as a multiple of each fund's target weight. A value of
        1.40 rebalances a fund once it exceeds 140% of its target, which for an
        evenly weighted four asset portfolio is Browne's 35% upper bound.
    rebal_per_low (float):
        Low rebalance band, as a multiple of each fund's target weight. A value of
        0.60 rebalances a fund once it falls below 60% of its target, which for an
        evenly weighted four asset portfolio is Browne's 15% lower bound.
    excel_export : bool
        If True, export data to Excel format.
    pickle_export : bool
        If True, export data to Pickle format.
    output_confirmation : bool
        If True, print confirmation message.
    target_weights (dict):
        Target portfolio weight for each fund, keyed by fund name, as decimals summing
        to 1.0 (for example {"Stocks": 0.30, "Bonds": 0.30, "Gold": 0.30, "Cash": 0.10}).
        If None, every fund is weighted equally, which is Browne's original 25% each.

    Returns:
    --------
    df (pd.DataFrame):
        DataFrame containing strategy data for all funds to be included. Also dumps the df to
    """
```

```
num_funds = len(fund_list)

# Weight each fund evenly unless explicit target weights were provided
if target_weights is None:
    target_weights = {fund: 1 / num_funds for fund in fund_list}
else:
    missing = [fund for fund in fund_list if fund not in target_weights]
    if missing:
        raise KeyError(f"target_weights is missing a weight for: {missing}")

    weight_total = sum(target_weights[fund] for fund in fund_list)
    if abs(weight_total - 1) > 1e-9:
        raise ValueError(
            f"target_weights for {fund_list} sum to {weight_total}, expected 1.0"
        )

df = close_prices_df.copy()
df.reset_index(inplace=True)

# Date to be used for annual rebalance
target_month = rebal_month
target_day = rebal_day

# Create a dataframe with dates from the specific month
rebal_date = df[df["Date"].dt.month == target_month]

# Specify the date or the next closest
rebal_date = rebal_date[rebal_date["Date"].dt.day >= target_day]

# Group by year and take the first entry for each year
rebal_dates_by_year = (
    rebal_date.groupby(rebal_date["Date"].dt.year).first().reset_index(drop=True)
)

"""
Column order for the dataframe:
df[fund + "_BA_Shares"]
df[fund + "_BA_$_Invested"]
df[fund + "_BA_Port_%"]
df['Total_BA_$_Invested']
df['Contribution']
df['Rebalance']
df[fund + "_AA_Shares"]
df[fund + "_AA_$_Invested"]
df[fund + "_AA_Port_%"]
df['Total_AA_$_Invested']
"""
```

```
# Calculate the columns and initial values for before action (BA) shares, $ invested, and
for fund in fund_list:
    df[fund + "_BA_Shares"] = (
        starting_cash * target_weights[fund] / df[fund + "_Close"]
    )
    df[fund + "_BA_$_Invested"] = df[fund + "_BA_Shares"] * df[fund + "_Close"]
    df[fund + "_BA_Port_%"] = target_weights[fund]

# Set column values initially
df["Total_BA_$_Invested"] = float(starting_cash)
df["Contribution"] = cash_contrib
df["Rebalance"] = "No"

# Set columns and values initially for after action (AA) shares, $ invested, and port %
for fund in fund_list:
    df[fund + "_AA_Shares"] = (
        starting_cash * target_weights[fund] / df[fund + "_Close"]
    )
    df[fund + "_AA_$_Invested"] = df[fund + "_AA_Shares"] * df[fund + "_Close"]
    df[fund + "_AA_Port_%"] = target_weights[fund]

# Set column value for after action (AA) total $ invested
df["Total_AA_$_Invested"] = float(starting_cash)

# Iterate through the dataframe and execute the strategy
for index, row in df.iterrows():

    # Ensure there's a previous row to reference by checking the index value
    if index > 0:

        # Initialize variable
        Total_BA_Invested = 0

        # Calculate before action (BA) shares and $ invested values
        for fund in fund_list:
            df.at[index, fund + "_BA_Shares"] = df.at[
                index - 1, fund + "_AA_Shares"
            ]
            df.at[index, fund + "_BA_$_Invested"] = (
                df.at[index, fund + "_BA_Shares"] * row[fund + "_Close"]
            )

            # Sum the asset values to find the total
            Total_BA_Invested = (
                Total_BA_Invested + df.at[index, fund + "_BA_$_Invested"]
            )

        # Calculate before action (BA) port % values
```

```
for fund in fund_list:
    df.at[index, fund + "_BA_Port_%"] = (
        df.at[index, fund + "_BA_$_Invested"] / Total_BA_Invested
    )

# Set column for before action (BA) total $ invested
df.at[index, "Total_BA_$_Invested"] = Total_BA_Invested

# Initialize variables
rebalance = "No"
date = row["Date"]

# Check for a specific date annually
# Simple if statement to check if date_to_check is in jan_28_or_after_each_year
if date in rebal_dates_by_year["Date"].values:
    rebalance = "Yes"
else:
    pass

# Check to see if any asset has drifted outside its rebalance band, which is
# set as a multiple of that asset's target weight, and if so set variable
for fund in fund_list:
    if (
        df.at[index, fund + "_BA_Port_%"]
        > target_weights[fund] * rebal_per_high
        or df.at[index, fund + "_BA_Port_%"]
        < target_weights[fund] * rebal_per_low
    ):
        rebalance = "Yes"
    else:
        pass

# If rebalance is required, rebalance back to each asset's target weight,
# else just divide the contribution across assets by their target weights
if rebalance == "Yes":
    df.at[index, "Rebalance"] = rebalance
    for fund in fund_list:
        df.at[index, fund + "_AA_$_Invested"] = (
            Total_BA_Invested + df.at[index, "Contribution"]
        ) * target_weights[fund]
else:
    df.at[index, "Rebalance"] = rebalance
    for fund in fund_list:
        df.at[index, fund + "_AA_$_Invested"] = (
            df.at[index, fund + "_BA_$_Invested"]
            + df.at[index, "Contribution"] * target_weights[fund]
        )
```

```
            # Initialize variable
            Total_AA_Invested = 0

            # Set column values for after action (AA) shares and port %
            for fund in fund_list:
                df.at[index, fund + "_AA_Shares"] = (
                    df.at[index, fund + "_AA_$_Invested"] / row[fund + "_Close"]
                )

                # Sum the asset values to find the total
                Total_AA_Invested = (
                    Total_AA_Invested + df.at[index, fund + "_AA_$_Invested"]
                )

            # Calculate after action (AA) port % values
            for fund in fund_list:
                df.at[index, fund + "_AA_Port_%"] = (
                    df.at[index, fund + "_AA_$_Invested"] / Total_AA_Invested
                )

            # Set column for after action (AA) total $ invested
            df.at[index, "Total_AA_$_Invested"] = Total_AA_Invested

        # If this is the first row
        else:
            pass

df["Return"] = df["Total_AA_$_Invested"].pct_change()
df["Cumulative_Return"] = (1 + df["Return"]).cumprod()

plan_name = "_".join(fund_list)

# Export to excel
if excel_export == True:
    df.to_excel(f"{plan_name}_Strategy.xlsx", sheet_name="data")
else:
    pass

# Export to pickle
if pickle_export == True:
    df.to_pickle(f"{plan_name}_Strategy.pkl")
else:
    pass

# Output confirmation
if output_confirmation == True:
    print(f"Strategy complete for {plan_name}")
else:
```

```
        pass

    return df
```

0.6.40 summary_stats

```
import pandas as pd
import numpy as np


def summary_stats(
    fund_list: list[str],
    df: pd.DataFrame,
    period: str,
    use_calendar_days: bool,
    excel_export: bool,
    pickle_export: bool,
    output_confirmation: bool,
) -> pd.DataFrame:
    """
    Calculate summary statistics for the given fund list and return data.

    Parameters
    ----------
    fund_list : list[str]
        List of funds. This is used below in the excel/pickle export but not in the analysis..
    df : pd.DataFrame
        Dataframe with return data. Assumes returns are in decimal format (e.g., 0.05 for 5%),
    period : str
        Period for which to calculate statistics. Options are "Monthly", "Weekly", "Daily".
    use_calendar_days : bool
        If True, use calendar days for calculations. If False, use trading days.
    excel_export : bool
        If True, export data to Excel format.
    pickle_export : bool
        If True, export data to Pickle format.
    output_confirmation : bool
        If True, print confirmation message.

    Returns
    -------
    pd.DataFrame
        DataFrame containing various portfolio statistics.
    """

    # Get the period in proper format
    period = period.strip().capitalize()
```

```python
# Map base timeframes
period_to_timeframe = {
    "Monthly": 12,
    "Weekly": 52,
    "Daily": 365 if use_calendar_days else 252,
}

try:
    timeframe = period_to_timeframe[period]
except KeyError:
    raise ValueError(
        f"Invalid period: {period}. Must be one of {list(period_to_timeframe.keys())}"
    )

df_stats = pd.DataFrame(df.mean(axis=0) * timeframe)  # annualized
df_stats.columns = ["Annual Mean Return (Arithmetic)"]
df_stats["Number of Observations"] = df.count()
data_start_date = df.index.min().strftime("%Y-%m-%d")
data_end_date = df.index.max().strftime("%Y-%m-%d")
df_stats["Data Start Date"] = data_start_date
df_stats["Data End Date"] = data_end_date
df_stats = df_stats[["Number of Observations", "Data Start Date", "Data End Date", "Annual
df_stats["Annualized Volatility"] = df.std() * np.sqrt(timeframe)  # annualized
df_stats["Annualized Sharpe Ratio"] = (
    df_stats["Annual Mean Return (Arithmetic)"] / df_stats["Annualized Volatility"]
)

df_cagr = (1 + df[df.columns[0]]).cumprod()
cagr = (df_cagr.iloc[-1] / 1) ** (1 / (len(df_cagr) / timeframe)) - 1
df_stats["CAGR (Geometric)"] = cagr

df_stats[f"{period} Max Return"] = df.max()
df_stats[f"{period} Max Return (Date)"] = df.idxmax().values[0]
df_stats[f"{period} Min Return"] = df.min()
df_stats[f"{period} Min Return (Date)"] = df.idxmin().values[0]

wealth_index = 1000 * (1 + df).cumprod()
previous_peaks = wealth_index.cummax()
drawdowns = (wealth_index - previous_peaks) / previous_peaks

df_stats["Max Drawdown"] = drawdowns.min()
df_stats["Peak"] = [
    previous_peaks[col][: drawdowns[col].idxmin()].idxmax()
    for col in previous_peaks.columns
]
df_stats["Trough"] = drawdowns.idxmin()

recovery_date = []
```

```
    for col in wealth_index.columns:
        prev_max = previous_peaks[col][: drawdowns[col].idxmin()].max()
        recovery_wealth = pd.DataFrame([wealth_index[col][drawdowns[col].idxmin() :]]).T
        recovery_date.append(
            recovery_wealth[recovery_wealth[col] >= prev_max].index.min()
        )
    df_stats["Recovery Date"] = recovery_date
    df_stats["Calendar Days to Recovery"] = (
        df_stats["Recovery Date"] - df_stats["Trough"]
    ).dt.days
    df_stats["MAR Ratio"] = df_stats["CAGR (Geometric)"] / -df_stats["Max Drawdown"]

    plan_name = "_".join(fund_list)

    # Export to excel
    if excel_export == True:
        df_stats.to_excel(f"{plan_name}_Summary_Stats.xlsx", sheet_name="data")
    else:
        pass

    # Export to pickle
    if pickle_export == True:
        df_stats.to_pickle(f"{plan_name}_Summary_Stats.pkl")
    else:
        pass

    # Output confirmation
    if output_confirmation == True:
        print(f"Summary stats complete for {plan_name}")
    else:
        pass

    return df_stats
```

0.6.41 yf_pull_data

```
import os
import pandas as pd
import yfinance as yf

from IPython.display import display


def yf_pull_data(
    base_directory: str,
    ticker: str,
    adjusted: bool,
    source: str,
```

```
    asset_class: str,
    excel_export: bool,
    pickle_export: bool,
    output_confirmation: bool,
) -> pd.DataFrame:
    """
    Download daily price data from Yahoo Finance and export it.

    Parameters:
    -----------
    base_directory : str
        Root path to store downloaded data.
    ticker : str
        Ticker symbol to download.
    adjusted : bool
        If True, download adjusted price data. If False, download both
        adjusted close and unadjusted open/high/low/close price data.
    source : str
        Name of the data source (e.g., 'Yahoo').
    asset_class : str
        Asset class name (e.g., 'Equities').
    excel_export : bool
        If True, export data to Excel format.
    pickle_export : bool
        If True, export data to Pickle format.
    output_confirmation : bool
        If True, print confirmation message.

    Returns:
    --------
    df : pd.DataFrame
        DataFrame containing the downloaded data.
    """

    # Download data from YF
    df = yf.download(ticker, start="1900-01-01", auto_adjust=adjusted, progress=False)

    # Drop the column level with the ticker symbol
    df.columns = df.columns.droplevel(1)

    # Reset index
    df = df.reset_index()

    # Remove the "Price" header from the index
    df.columns.name = None

    # Reset date column
    df["Date"] = df["Date"].dt.tz_localize(None)
```

```
    # Set 'Date' column as index
    df = df.set_index("Date", drop=True)

    # Drop data from last day because it's not accurate until end of day
    df = df.drop(df.index[-1])

    # Create directory
    directory = f"{base_directory}/{source}/{asset_class}/Daily"
    os.makedirs(directory, exist_ok=True)

    # Export to excel
    if excel_export == True:
        df.to_excel(f"{directory}/{ticker}.xlsx", sheet_name="data")
    else:
        pass

    # Export to pickle
    if pickle_export == True:
        df.to_pickle(f"{directory}/{ticker}.pkl")
    else:
        pass

    # Output confirmation
    if output_confirmation == True:
        print(f"The first and last date of data for {ticker} is: ")
        display(df[:1])
        display(df[-1:])
        print(f"Yahoo Finance data complete for {ticker}")
        print(f"--------------------")
    else:
        pass

    return df
```

0.7 References

None